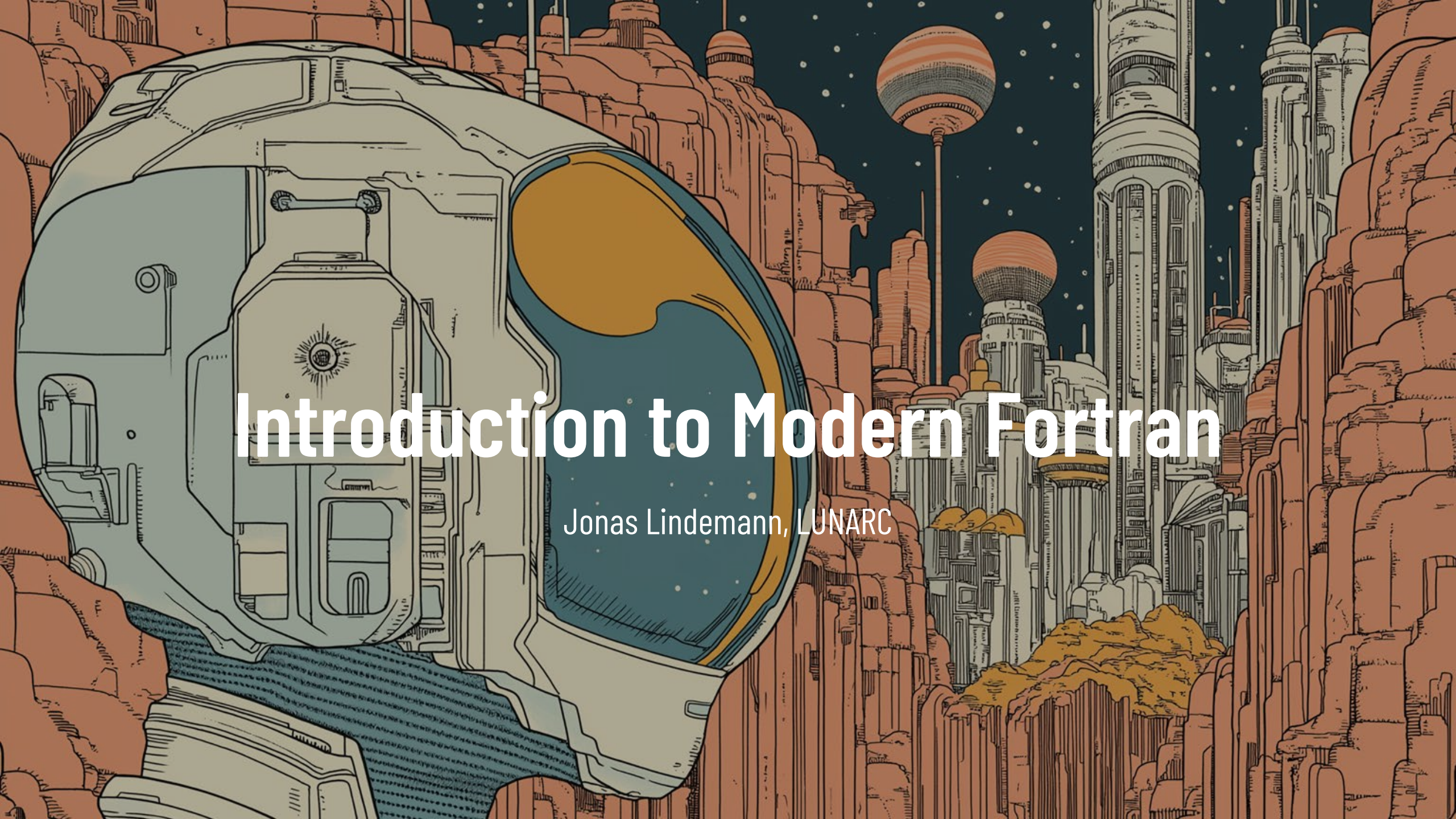


Introduction to Modern Fortran

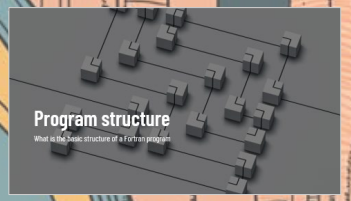
Jonas Lindemann, LUNARC

Introduction to Modern Fortran

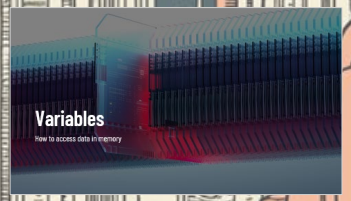






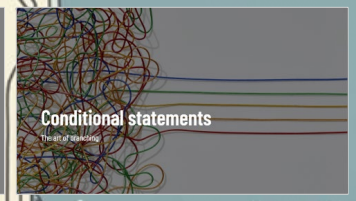


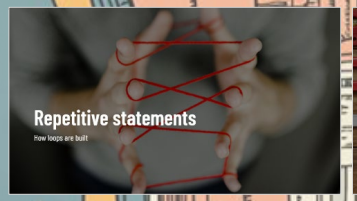


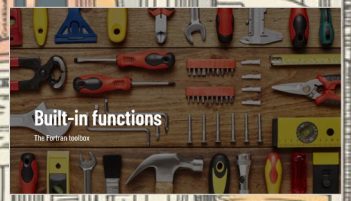




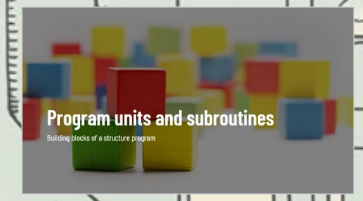


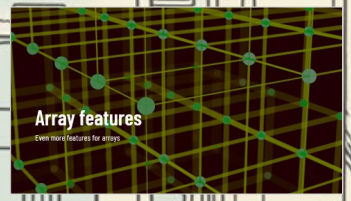


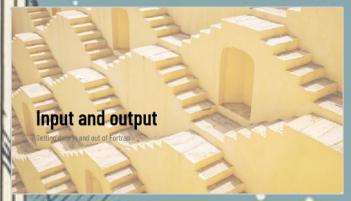




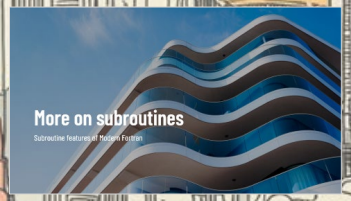


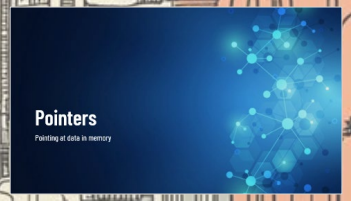


















Why Modern Fortran

Advantages of a standardized language

Why Modern Fortran

- Standardised language continuously Developed
  - Latest version Fortran 2023
- Backwards compatible. Can link with Fortran 77 code
- Optmised for high-performance computing
- An array is a native data type
  - Most built-in functions work on arrays by default
  - Standard operators also work on arrays
- Modules for structuring and maintaining code
- Fully object-oriented – Extension to derived data types
- Perfect language for creating Python extensions (F2PY)



History

One of the oldest programming languages

Fortran development

- First version 1954
- FORTRAN 66
- FORTRAN 77
- Fortran 90 – Backwards compatible
- (The F programming language)
- Fortran 95
- Fortran 2003, (2004)
- Fortran 2008, (2010)
- Fortran 2018 (Minor revision)
- Fortran 2023 (Minor revision

Pascal

```
program tpk(input,output);
var
  i : integer;
  y : real;
  a : array [0..10] of real;

function f ( t : real) : real;
begin
  f := sqrt(abs(t)) + 5*t*t*t
end;

begin
  for i := 0 to 10 do read(a[i]);
  for i := 10 downto 0 do
  begin
    y := f(a[i]);
    if y  >  400 then
      writeln(i,' TOO LARGE')
    else
      writeln(i,y);
    end;
  end;
end.
```

C

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

double f (double t)
{
   double temp;
   temp = sqrt(fabs(t)) + 5*pow(t,3);
   return temp;
}

int main()
{
   int i;
   double y;
   double a[11];
   for ( i = 0; i <= 10; ++i)
      scanf("%lf", &a[i]);
   for ( i = 10; i >= 0; i = i - 1 ) {
      y = f(a[i]);
      if ( y > 400 ) {
         printf(" %d",i);
         printf(" TOO LARGE\n");
      }
      else {
         printf(" %d",i);
         printf(" %lf",y);
         printf(" \n");
      }
   }
   return 0;
}
```

Fortran 0

```
      DIMENSION A(11)
      READ A
2     DO 3,8,11 J=1,11
3     I=11-J
      Y=SQRT(ABS(A(I+1)))+5*A(I+1)**3
      IF (400>=Y) 8,4
4     PRINT I,999.
      GOTO 2
8     PRINT I,Y
11    STOP
```

Fortran I

```
C       FORTRAN I STYLE
        FUNF(T)=SQRTF(ABSF(T))+5.0*T**3
        DIMENSION A(11)
    1   FORMAT(6F12.4)
        READ 1,A
        DO 10 J=1,11
        I=11-J
        Y=FUNF(A(I+1))
        IF(400.0-Y)4,8,8
    4   PRINT 5,I
    5   FORMAT(I10,10H TOO LARGE)
        GOTO 10
    8   PRINT 9,I,Y
    9   FORMAT(I10,F12.7)
   10   CONTINUE
        STOP 52525
```

Fortran 77

```
      PROGRAM TPK
C     FORTRAN 77 STYLE
      REAL A(0:10)
      READ (5,*) A
      DO 10 I = 10, 0, -1
            Y = FUN(A(I))
            IF ( Y .LT. 400) THEN
                  WRITE(6,9) I, Y
    9             FORMAT(I10, F12.6)
            ELSE
                  WRITE (6,5) I
    5             FORMAT(I10,' TOO LARGE')
            ENDIF
   10 CONTINUE
      END

      REAL FUNCTION FUN(T)
      REAL T
      FUN = SQRT(ABS(T)) + 5.0*T**3
      END
```

Modern Fortran 9x/20xx

```
program sample1

    integer, parameter :: ap=selected_real_kind(15,300)
    real(kind=ap) :: x, y
    real(kind=ap) :: k(20,20)

    x = 6.0_ap
    y = 0.25_ap

    write(*,*) x
    write(*,*) y
    write(*,*) ap

    call myproc(k)

    write(*,*) k(1,1)

contains

subroutine myproc(k)

    integer, parameter :: ap=selected_real_kind(15,300)
    real(kind=ap) :: k(20,20)

    k=0.0_ap
    k(1,1) = 42.0_ap

    return

end subroutine myproc

end program sample1
```



Source form

The different flavors of Fortran source

Source form

	Free form	Fixed form
Source format	Max 132 characters	Code in positions 7-72, line numbers in position 2-5
Comments	! anywhere on a line	!, C or * at position 1
Continuation	& at end of line	character at position 6 on continuation line
Multiple statements per line	; between statements	On statement per row

Example of Fortran code

```
program example

     implicit none

     integer, parameter ::  rk=selected_real_kind(15,300)
     real(rk) :: x,y
     real(rk) :: K(20,20)

     x = 6.0_rk   ! This is line comment

     y = &
          0.25_rk + &
          0.25_rk

     write(*,*) x; write(*,*) y
     write(*,*) ap

     call myproc(K)

     stop

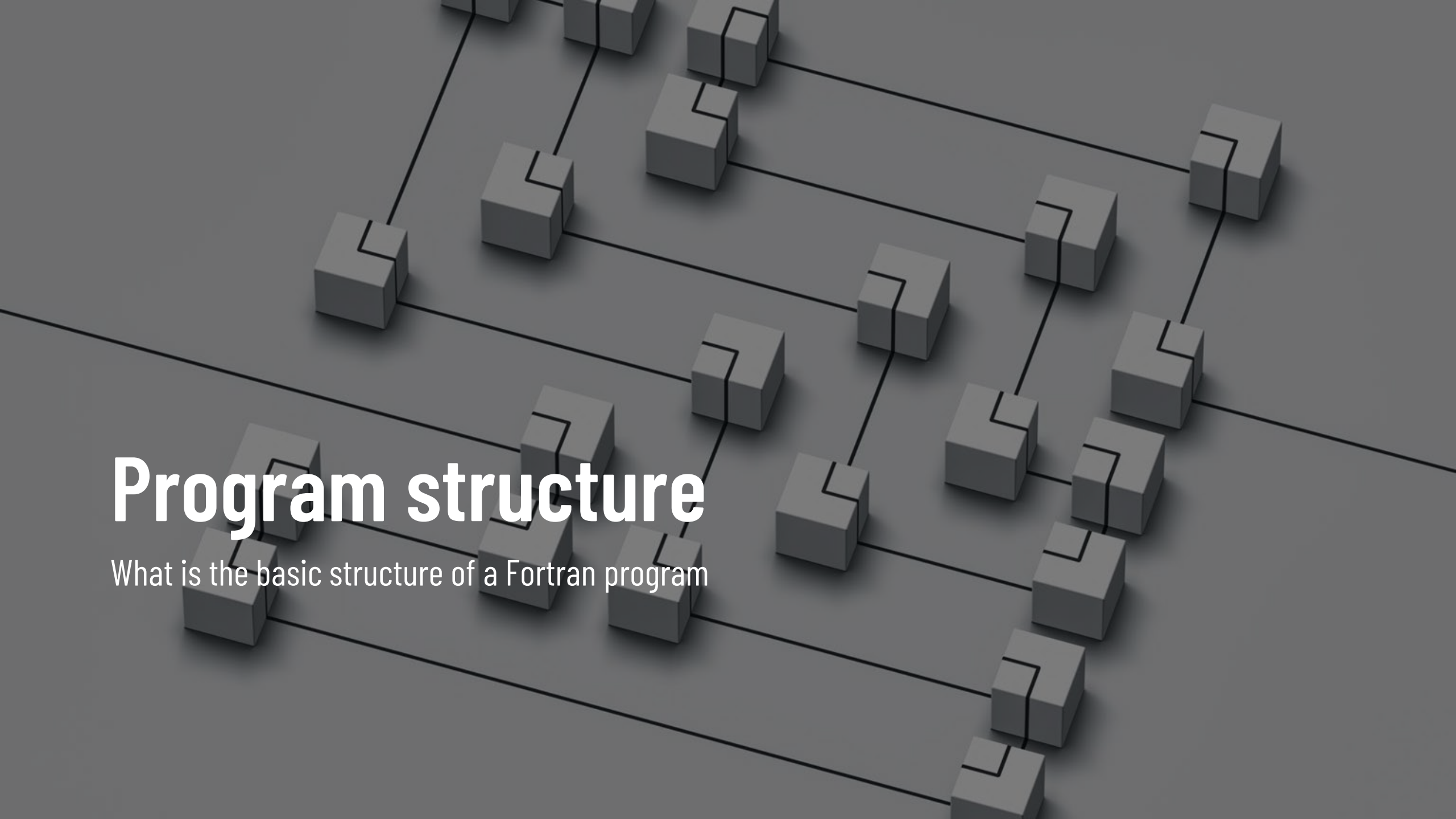
contains

    subroutine myproc(K)

        real(rk) :: K(:,:)
        K = 0.0_rk

    end subroutine

end program example
```



Program structure

What is the basic structure of a Fortran program

Program structure

[**program** program-name]
[specication statements]
[executable statements]
[**contains**]
[subroutines]
end [**program** [program-name]]



Compiling and running code

Going from source to executable

What is a program / executable



Executable code - machine code



Links to system libraries for interacting with the operating system and file system



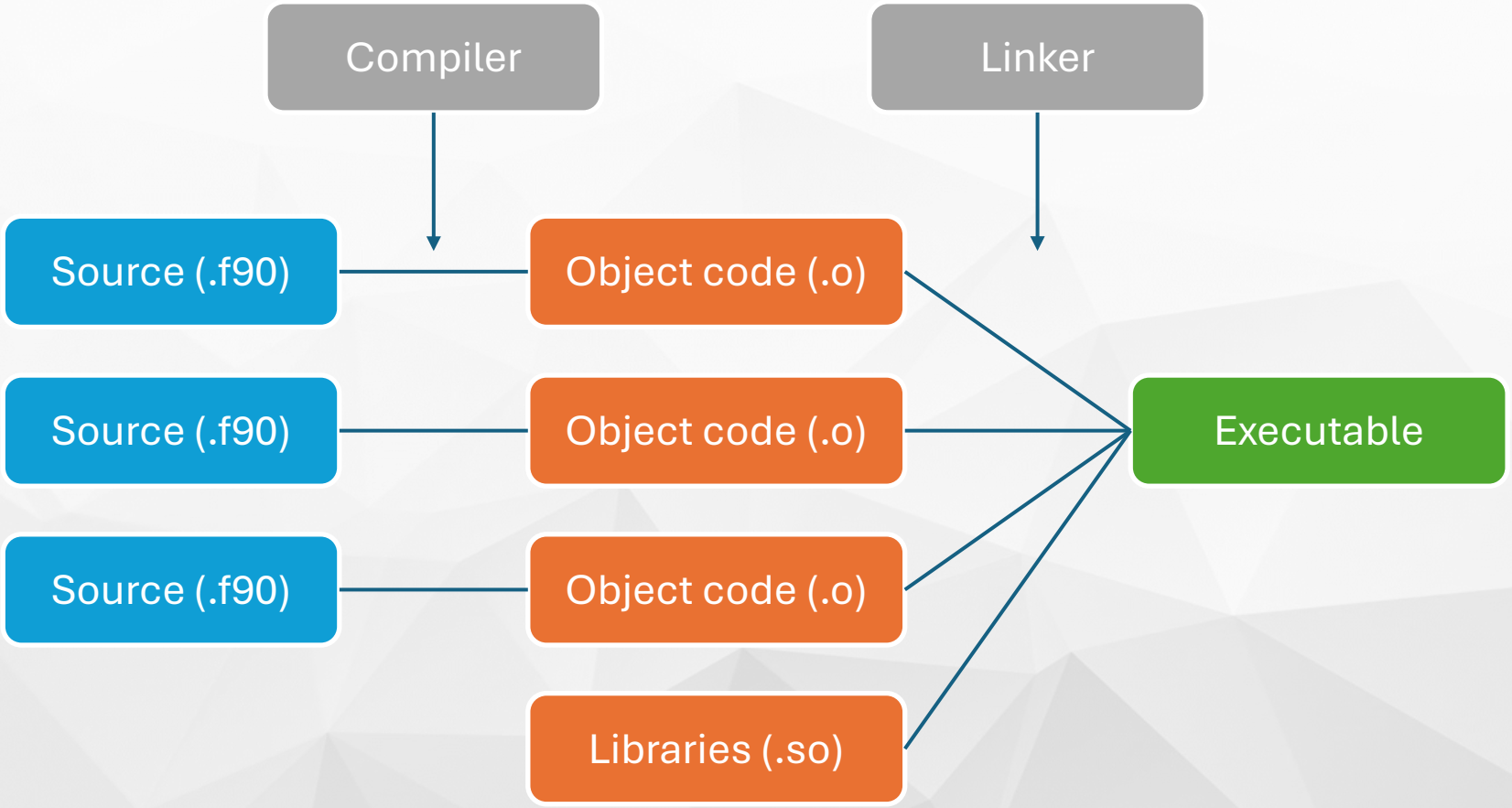
Links to external libraries

How to build a program / executable

- Handcoding machine code (HARD)
- Writing assembler source code (hard)
- Using Fortran/C/C++ or any other higher language (easier)
- Using Python (easiest)

Creating an executable from a high level language

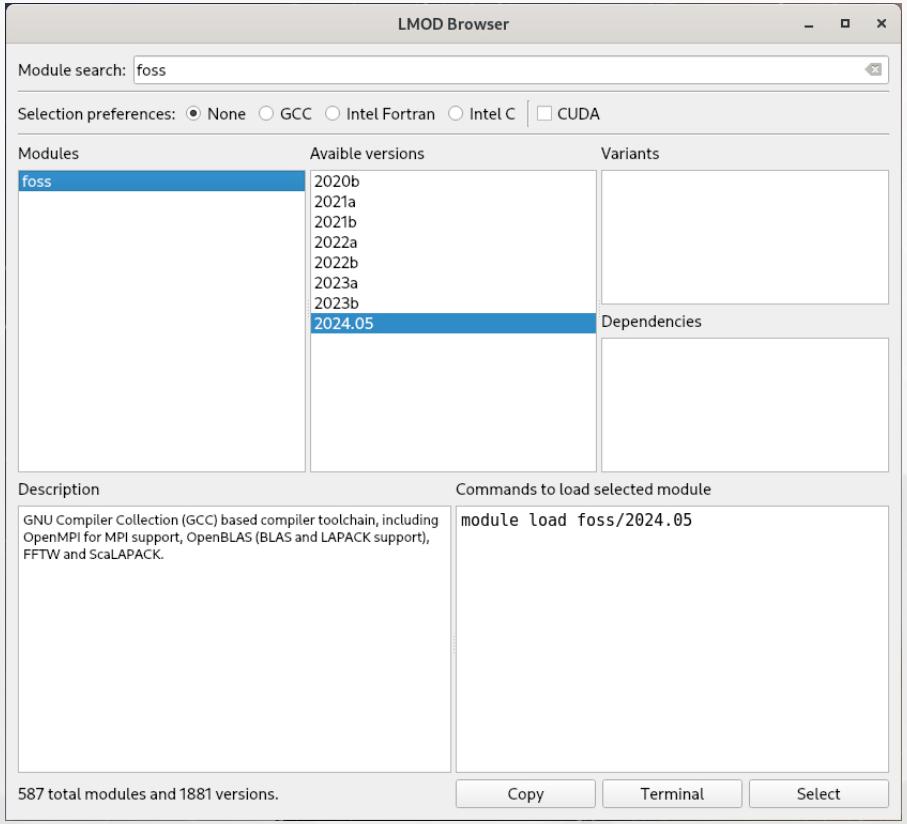
1. Write source code
2. Compile source code to object files (unlinked machine code)
3. Link object files and libraries to an executable


Compiler
Linker
Source (.f90)
Source (.f90)
Source (.f90)
Object code (.o)
Object code (.o)
Object code (.o)
Libraries (.so)
Executable

Compiling on COSMOS

- Fortran compilers are usually found in toolchain modules
- GNU compilers found in the **foss/20xx** modules
- Intel compilers found in **intel/20xx** or **intel-compilers/20xx**

Getting out our tools




```
$ ml foss/2024.05
$ ml

Currently Loaded Modules:
  1) SoftwareTree/Milan  (S)  13) UCX/1.16.0
  2) GCCcore/13.3.0           14) libfabric/1.21.0
  3) zlib/1.3.1               15) PMIx/5.0.2
  4) binutils/2.42            16) PRRTE/3.0.5
  5) GCC/13.3.0               17) UCC/1.3.0
  6) numactl/2.0.18           18) OpenMPI/5.0.3
  7) XZ/5.4.5                 19) OpenBLAS/0.3.27
  8) libxml2/2.12.7           20) FlexiBLAS/3.4.4
  9) libpciaccess/0.18.1      21) FFTW/3.3.10
 10) hwloc/2.10.0             22) FFTW.MPI/3.3.10
 11) OpenSSL/3                23) ScaLAPACK/2.2.0-fb
 12) libevent/2.1.12          24) foss/2024.05

  Where:
   S:  Module is Sticky, requires --force to unload or purge



$ gfortran --version
GNU Fortran (GCC) 13.3.0
```

Compiling Fortran code

- Write source code - use .f90 extension
- Compile a Fortran source to an object file with:
  - **`gfortran -c mysource.f90`**
  - generates the object file **mysource.o**
- multiple source files can be given
  - **`gfortran -c source1.f90 source2.f90`**
  - generates the object files **source1.o** and **source2.o**

Linking Fortran code

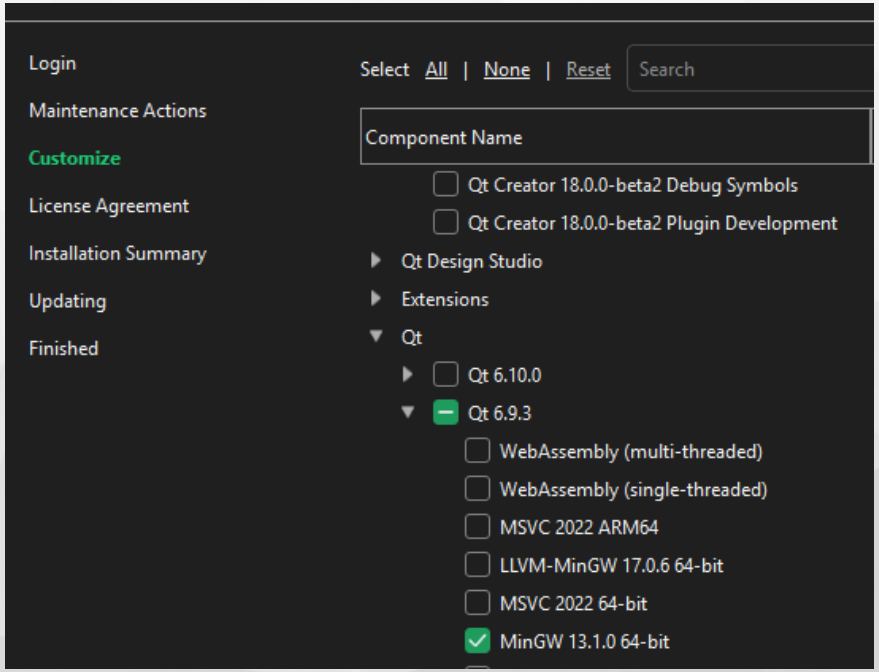
- Linking is done using
  - `gfortran mysource.o`
  - generates an executable **a.out**
- Executable can be given a name
  - `gfortran mysource.o -o myexec`
  - generates an executable **myexec**

Compiling and linking in a single step

- Compiling and linking can be done in a single step
  - `gfortran mysource.f90 -o myexec`
- Multiple source files can be compiled in a single step
  - `gfortran mysource.f90 source1.f90 source2.f90 -o myexec`

Using Qt Creator for Fortran Development

- You can install a complete Fortran Development environment using Qt Creator
- https://modern-fortran-in-science-and-technology.readthedocs.io/en/latest/chapter_qtcreator.html
- https://scientific-programming-in-python-and-fortran.readthedocs.io/en/latest/fortran.html
- Make sure you install the mingw32 toolchain.





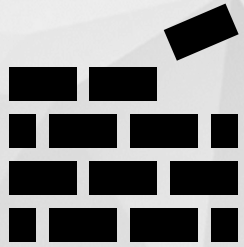
Variables

How to access data in memory

Variables



Variables are **named storage locations** in memory that hold values that can be modified during the program execution



Compiler allocates memory for the variables according to the data type of the variable

Naming of variables

- 1-31 alphanumeric characters (letters, numbers, and underscore)
- Variable names can consist of both upper-case and lower-case characters.
- Variables are case insensitive **A** and **a** are references to the same variable
- The first character can't be a number

Example of variable names

Valid variable names
a
a_thing
x1
mass
q123
time_of_flight

Invalid variable names	
1a	First char a number
a thing	Contains a space
_	Only non-alphanumeric char

Variable types

- **integer**, integers
- **real**, floating point numbers
- **complex**, complex numbers
- **logical**, boolean variables
- **character**, strings and characters
- **derived types** (structs)

Variable declaration

type [[,**attribute**]... ::] entity-list

Variable declaration

```
integer :: a ! Scalar integer variable
real :: b ! Scalar floating point variable
logical :: flag ! boolean variable

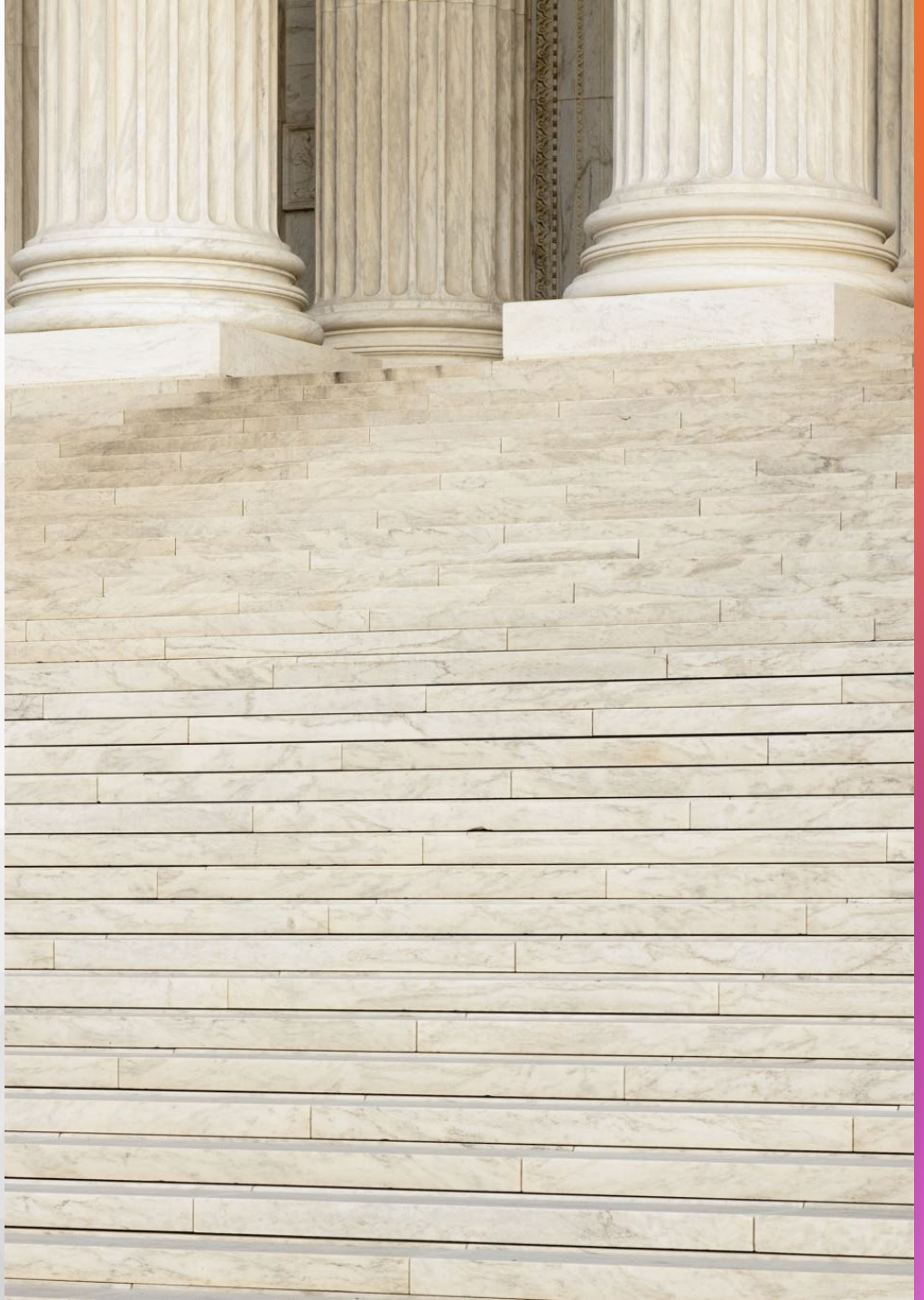
real :: D(10) ! Floating point array consisting of 10
elements
real :: K(20,20) ! Floating point array of 20x20 elements

integer, dimension(10) :: C ! Integer array of 10 elements

character :: ch ! Character
character, dimension(60) :: chv ! Array of characters
character(len=80) :: line ! Character string
character(len=80) :: lines(60) ! Array of strings
```

Constants

- Named values that can’t be changed after they have been defined
- Compiler enforces the immutability of the constants
- In Fortran constants are declared just as variables with an additional attribute, **parameter**, in the declaration



Variable precision

- Precision can be specified as an extra parameter for the data type
- Precision specifiers are dependent on compiler and system architecture
- special directives can be used to specify precision independent of architecture
  - selected_real_kind(...)
  - selected_int_kind(...)

Variable precision

Explicit precision specifiers

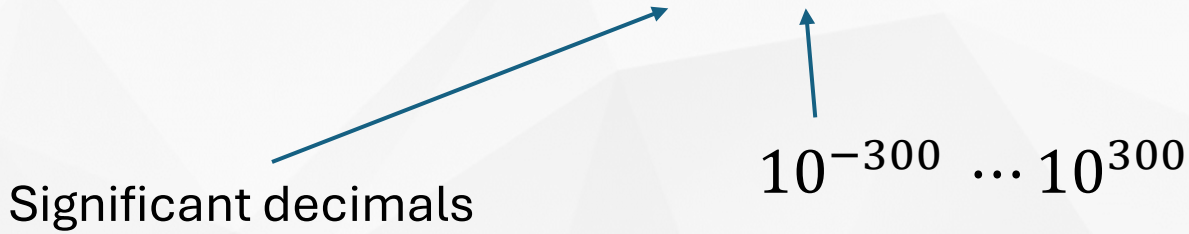
```
real(8) :: A
real(4) :: B
integer(4) :: I
```

Using selected_real_kind

```
integer, parameter :: rk = selected_real_kind(15,300)
real(kind=rk) :: X,Y
```

selected_real_kind(...)

```
integer, parameter :: rk = selected_real_kind(15,300)
```



Variable declaration with precision specifier

```
X = 6.2_rk
```

Example

```
program constants

    implicit none

    integer, parameter :: ap = selected_real_kind(15,300)
    real(ap) :: pi1, pi2

    pi1 = 3.141592653589793
    pi2 = 3.141592653589793_ap

    write(*,*) 'pi1 = ', pi1
    write(*,*) 'pi2 = ', pi2

    stop

end program constants
```

```
pi1 = 3.14159274101257
pi2 = 3.14159265358979
```

General type rule

- By default, variables does not have to be declared in Fortran
- The general type rule defines
  - variables starting with **I..N** are **integer**
  - other variables are **real** by default
- This can lead to difficult bugs and should be avoided
- By placing the directive **implicit none** at the beginning of a program or module forces declaration of all variables.

Variable assignment

`real(rk)`	`a = 42.0_rk`
`integer(ik)`	`b = 42_ik`
`logical`	`c = .false.` `d = .true.`

`character`	`first_name = 'Jan'`
	`last_name = 'Johansson'`
	`company_name1 = "McDonals's"`
	`company_name2 = 'McDonald"s'`

Example

Defined and undefined variables

- A variable is said to be undefined until it has been assigned a value
  - `real :: speed ! variable speed exists, but is undefined`
- An undefined variable must not be referenced
- When a variable is assigned a value it is considered defined
  - `speed = 42.0 ! variable speed is defined`
  - It can now be referenced by other expressions
- An array is said to be defined when all elements have been assigned values

Derived data types

- In some cases, it can be beneficial to define your own datatypes
  - Derived data types
- Can contain
  - Standard variable types
  - Arrays
  - Other derived data types
- A derived data type is defined by a type-statement block listing the included data members and type
- Declared using the type keyword
  - **type**([data type]) :: variable

Derived data types

```
program derived1

    use mf_utils

    implicit none

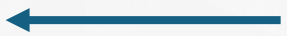
    type particle
        real(dp) :: x
        real(dp) :: y
        real(dp) :: z
        real(dp) :: m
    end type particle

    type(particle) :: p

    p % x = 0.0_dp
    p % y = 1.0_dp
    p % z = 2.0_dp
    p % m = 3.0_dp

    print*, p % x
    print*, p % y
    print*, p % z
    print*, p % m

end program derived1
```



data type definition



variable declaration



assignment using % operator

data access with % operator

Derived data types

```fortran
program derived2

    use mf_utils

    implicit none

    type data_record
        real(dp) :: a
        integer  :: b
        logical  :: c
        character(10) :: name
    end type data_record

    type(data_record) :: d

    d % a = 0.0_dp
    d % b = 1
    d % c = .true.
    d % name = 'fortran'

    print*, d % a
    print*, d % b
    print*, d % c
    print*, d % name

end program derived2
```



Mixed data types

Data types with iso_c_binding

- To increase the operability with C and C++ Fortran has added a special module defining C data types for use with Fortran
- These identifiers can be used in variables declarations
- Some of the data types defined are
  - **c_float**
  - **c_double**
  - **c_int**
  - **c_long**
- A c_float variable in Fortran can be declared with
  - `real(c_float) :: x`

Data types with iso_c_binding

```
program c_interop

    use iso_c_binding

    implicit none

    integer(c_int) :: a
    real(c_float) :: b
    real(c_double) :: c

    a = 42
    b = 42.0_c_float
    c = 84.0_c_double

    print *, a, b, c

end program c_interop
```



Operators and expressions

Formulating expressions in Fortran

Arithmetic operators

**	power of
*	multiplication
/	division
+	addition
-	subtraction

Order of operations

1. Parentesis ( )
2. Operations with ** (power of)
3. Operations with * or /
4. Operations with + or -

```
c = a+b/2   ! is equivalent to a + (b/2)
c = (a+b)/2 ! in this case (a + b) is evaluated and then /2
```

Relational operators

Fortran 90 or higher	Old operator	Meaning
<	.lt.	less than
<=	.le.	less than or equal to
>	.gt.	greater than
>=	.ge.	greater than or equal to
==	.eq.	equal to
/=	.ne.	not equal to

Logical operators

Operator	Meaning
.and.	and
.or.	or
.not.	not

Integer expressions

- Results of divisions will be truncated towards 0
  - 6/3 = 2
  - 8/3 = 2
  - -8/3 = -2
- WARNING
  - 2**3 = 8
  - 2**(-3) = 1/(2**3) = 0
- https://godbolt.org/z/6j6h7s6Pd
- https://godbolt.org/z/cjqr9Yvd6

Mixed mode expressions

- Numeric expressions with operands of differing datatypes
- Weaker of to datatypes will be coerced to the stronger one
- Result will be of the stronger type

```
real :: a
integer :: i
real :: b

b = a*i
```



Coerced to real

Array expressions

- Intrinsic operations can also apply to arrays
- Same-shape arrays are assumed
- One or more operands can be scalar
  - scalar values "broadcast" to array operands
- Order of array operations is not specified in the standard
  - Enabling efficient execution on a vector or parallel computer



Array expressions

```
real, dimension(10,20) :: a, b

real, dimension(5) :: v

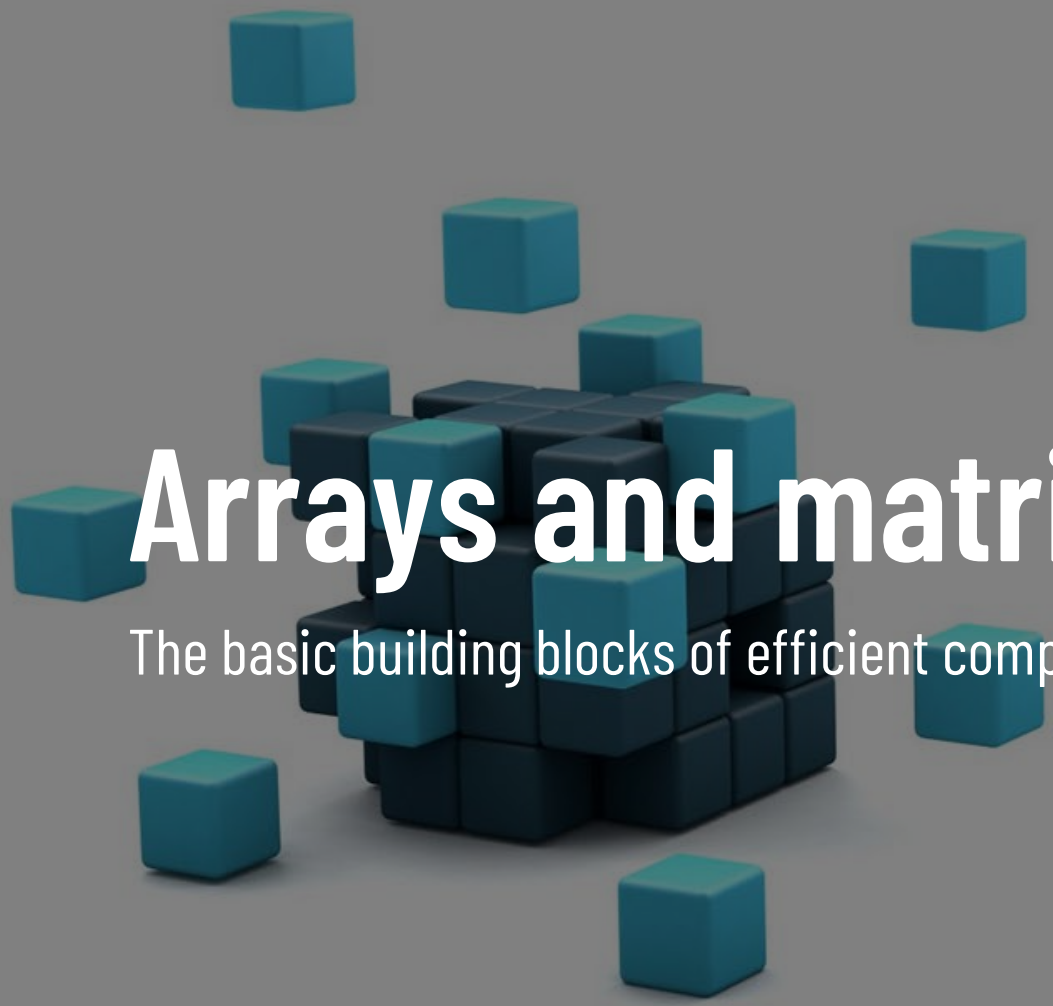
a/b             ! Array of shape (10,20), with elements a(i,j)/b(i,j)

v+1.            ! Array of shape (5), with elements v(i) + 1.0

5/v+a(1:5,5) ! Array of shape (5), with elements 5/v(i) + a(i,5)

a.eq.b          ! .true. if a(i,j)==b(i,j) and .false. otherwise
```

https://godbolt.org/z/h6o6xK7Wc



Arrays and matrices

The basic building blocks of efficient computing

Arrays and matrices

- Important concepts in scientific and technical applications
- Efficient datatype in Fortran
- Can be statically or dynamically allocated (Fortran 90)
- Supports advanced indexing, slicing and assignment

```
integer, parameter :: rk = selected_real_kind(15,300)

real(rk), dimension(20,20) :: K ! Matrix 20x20 elements
real(rk) :: Ke(8,8)             ! Matrix 8x8 elements
real(rk) :: fe(6)               ! Array with 6 elements
```

Array indices

- By default starting index of an array is 1
- Can be redefined

```
! Array with indices
!
! [-3, -2, -1, 0, 1, 2, 3]

real(rk) :: idx(-3:3)
```

Array assignment

Individual elements

```
K(5,6) = 5.0
```

Entire array

```
K = 5.0
```

Multiple values in single assignment

```
real(rk) :: v(5) ! Array with 5 elements

v = (/ 1.0, 2.0, 3.0, 4.0, 5.0 /)
```

Assigning slices

```
program index_notation

    implicit none

    real :: A(4,4)
    real :: B(4)
    real :: C(4)

    B = A(2,:) ! Assigns B the values of row 2 in A
    C = A(:,1) ! Assigns C the values of column 1 in A

    stop

end program index_notation
```

Assigning slices

Using slices rows and columns can be assigned in single statements

```
! Assign row 5 in matrix K the values 1, 2, 3, 4, 5

K(5,:) = (/ 1.0, 2.0, 3.0, 4.0, 5.0 /)

! Assign the array v the values 5, 4, 3, 2, 1

v = (/ 5.0, 4.0, 3.0, 2.0, 1.0 /)
```

Array storage

- Arrays (1, 2 .. more dimensions) are always stored contiguous in memory
- twodimensional arrays are stored columnwise in memory

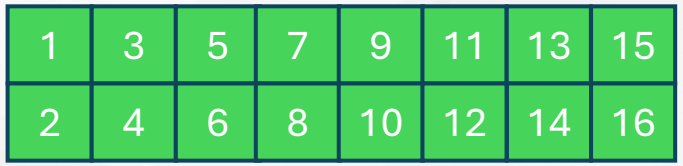
Array storage

```
real :: A(16)
```






```
real :: A(8,2)
```




```
real :: A(2,8)
```

Assigning slices

```
! Assign row 5 in matrix K the values 1, 2, 3, 4, 5 - K(5,5)

K(5,:) = (/ 1.0, 2.0, 3.0, 4.0, 5.0 /)

! Assign the array v the values 5, 4, 3, 2, 1

v = (/ 5.0, 4.0, 3.0, 2.0, 1.0 /)
```



Arrays and derived data types

```
program derived2

    use mf_utils

    implicit none

    type data_record
        real(dp) :: a
        integer  :: b
        logical  :: c
        character(10) :: name
    end type data_record

    type(data_record) :: d(5)
    integer :: i

    do i=1,5
        d(i) % a = 0.0_dp
        d(i) % b = i
        d(i) % c = .true.
        d(i) % name = 'fortran'
    end do

    do i=1,5
        print*, d(i) % b
    end do

end program derived2
```



derived data type array



Conditional statements

The art of branching

if-statements

if-statement

```
if (scalar-logical-expr) statement
```

```
if (scalar-logical-expr) then
      block
end if
```

```
if (scalar-logical-expr1) then
      block1
else if (scalar-logical-expr2) then
      block2
else
      block3
end if
```

if-statement

```
program logic

    implicit none
    integer :: x
    logical :: flag

    write(*,*) 'Enter an integer value.'
    read(*,*) x

    flag = .false.
    if (x>1000) then
        x = 1000
        flag = .false.
    end if

    if (x<0) then
        x = 0
        flag = .false.
    end if

    if (flag) then
        write(*,'(a,I4)') 'Corrected value = ', x
    else
        write(*,'(a,I4)') 'Value = ', x
    end if

end program logic
```

select statement

select statement

```
select case (expr)
    case selector
        block
end select
```

```
select case (expr)
    case selector
        block
    case default
        block
end select
```

Example

select statement

```
program case_sample
    integer :: value
    write(*,*) 'Enter a value'
    read(*,*) value

    select case (value)
        case (:0)
            write(*,*) 'Greater than one.'
        case (1)
            write(*,*) 'Number one!'
        case (2:9)
            write(*,*) 'Between 2 and 9.'
        case (10)
            write(*,*) 'Number 10!'
        case (11:41)
            write(*,*) 'Less than 42 but greater than 10.'
        case (42)
            write(*,*) 'Meaning of life or perhaps 6*7.'
        case (43:)
            write(*,*) 'Greater than 42.'
        case default
            write(*,*) 'This should never happen!'
    end select

end program case_sample
```



Repetitive statements

How loops are built

Example

do-statements

start-value end-value

```
do i = 1, 10
    j = j + i
end do
```

i → | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

start-value end-value step

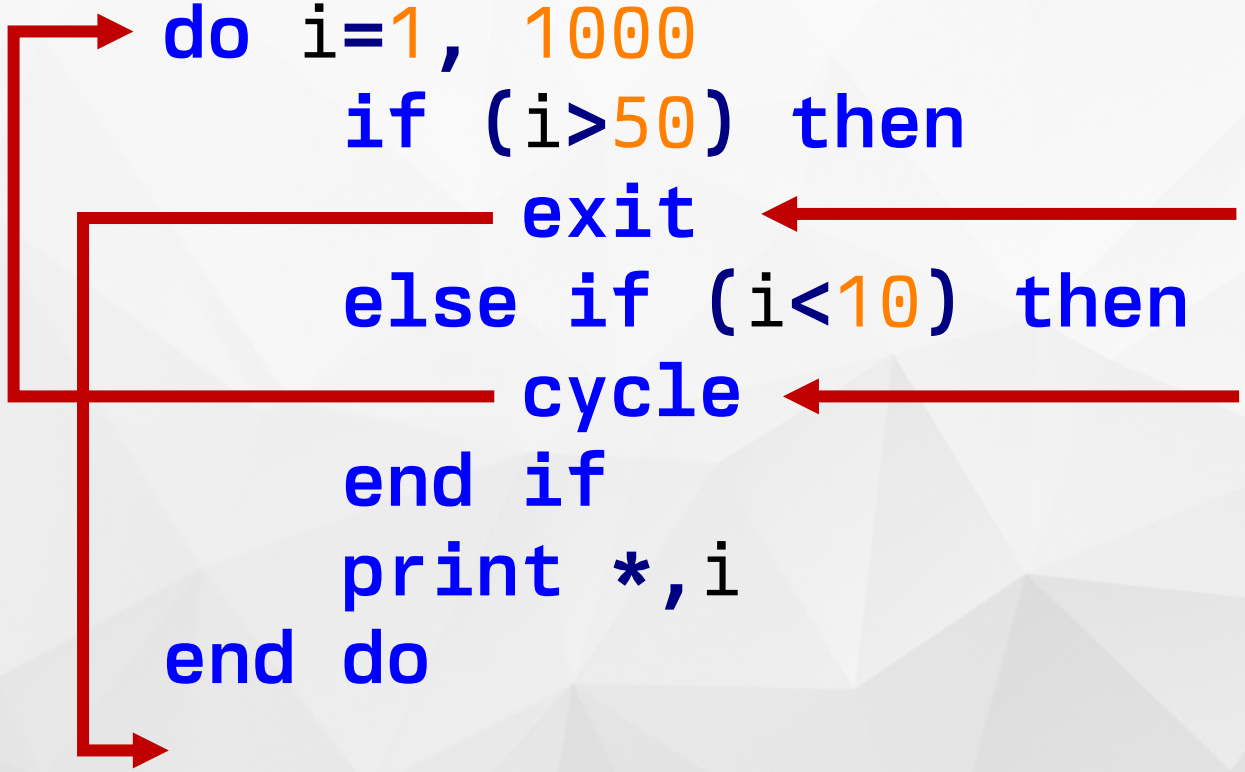
```
do i = 2, 30, 4
    j = j + i
end do
```

i → | 2 | 6 | 10 | 14 | 18 | 22 | 26 | 30 |

loop-variable is not allowed to be modified inside a do-statement

Example

do-statements



Exit do-statement and execute next executable statement after loop.

Continue to next iteration

10
11
12
.
.
.
47
48
49
50

do-while statements

Execute until a certain condition is fulfilled

```
do while (scalar-logical-expr)
      block
end do
```

Example

do-while statements

```
x = 0.0
do while (x<1.05)
     f = sin(x)
     x = x + 0.1
     write(*,*) x, f
end do
```

```
 0.100000001        0.00000000
 0.200000003        9.98334214E-02
 0.300000012       0.198669329
 0.400000006       0.295520216
 0.500000000       0.389418334
 0.600000024       0.479425550
 0.700000048       0.564642489
 0.800000072       0.644217730
 0.900000095       0.717356145
  1.00000012       0.783326983
  1.10000014       0.841471076
```



Built-in functions

The Fortran toolbox

Mathematical functions

Function	Description
acos(x)	Returns $\arccos(x)$
asin(x)	Returns $\arcsin(x)$
atan(x)	Returns $\arctan(x)$
atan2(y,x)	Returns $\arctan(\frac{y}{x})$ from $-\pi$ till $-\pi$
cos(x)	Returns $\cos(x)$
cosh(x)	Returns $\cosh(x)$
exp(x)	Returns e^x
log(x)	Returns $\ln(x)$
log10(x)	Returns $\lg(x)$
sin(x)	Returns $\sin(x)$
sinh(x)	Returns $\sinh(x)$
sqrt(x)	Returns $\sqrt{x}$
tan(x)	Returns $\tan(x)$
tanh(x)	Returns $\tanh(x)$

Example

Conversion functions

Function	Description
abs(a)	Returns absolute value of a
aint(a)	Truncates a floating point value
int(a)	Converts a floating point value to an integer
nint(a)	Rounds a floating point value to the nearest integer
real(a)	Converts an integer to a floating point value
max(a1,a2[,a3 ,...])	Returns the maximum value of two or more values
min(a1,a2[,a3 ,...])	Returns the minimum value of two or more values

Vector and matrix functions

Function	Description
dot_product(u, v)	Returns the scalar product of $u \cdot v$
matmul(A, B)	Matrix multiplication. The result must have the same for as $\mathbf{AB}$
transpose(C)	Returns the transpose $\mathbf{C}^T$. Elementet C_{ij}^T motsvarar C_{ji}

Vector and matrix functions

Functions and operators can be used with arrays.

```
real(rk), dimension(20,20) :: A, B, C

C = A/B      ! Division Cij = Aij/Bij
C = sqrt(A) ! Square root Cij = sqrt(Aij)
```



Example

Using the Fortran toolbox

A stiffness matrix

$$\mathbf{K}_e = (\mathbf{G}^T \mathbf{K}_{el})\mathbf{G}$$

$$\mathbf{G} = \begin{bmatrix} n_x & n_y & n_z & 0 & 0 & 0 \\ 0 & 0 & 0 & n_x & n_y & n_z \end{bmatrix}$$

$$\mathbf{K}_{el} = \frac{EA}{L} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

$$L = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

$$n_x = \frac{x_2 - x_1}{L}, \quad n_y = \frac{y_2 - y_1}{L}, \quad n_z = \frac{z_2 - z_1}{L}$$

Example

Fortran implementation

```
program function_sample
implicit none

integer, parameter :: rk = selected_real_kind(15,300)

integer :: i, j

real(rk) :: x1, x2, y1, y2, z1, z2
real(rk) :: nx, ny, nz
real(rk) :: L, E, A
real(rk) :: Kel(2,2)
real(rk) :: Ke(6,6)
real(rk) :: G(2,6)

! Initiate scalar values

E = 1.0_rk
A = 1.0_rk
x1 = 0.0_rk
x2 = 1.0_rk
y1 = 0.0_rk
y2 = 1.0_rk
z1 = 0.0_rk
z2 = 1.0_rk
```

Fortran implementation ...

```
! Calcuate directional cosines

L = sqrt( (x2-x1)**2 + (y2-y1)**2 + (z2-z1)**2 )
nx = (x2-x1)/L
ny = (y2-y1)/L
nz = (z2-z1)/L

! Calucate local stiffness matrix

Kel(1,:) = (/ 1.0_ap , -1.0_ap /)
Kel(2,:) = (/ -1.0_ap, 1.0_ap /)
Kel = Kel * (E*A/L)

G(1,:) = (/ nx, ny, nz, 0.0_ap, 0.0_ap, 0.0_ap /)
G(2,:) = (/ 0.0_ap, 0.0_ap, 0.0_ap, nx, ny, nz /)

! Calculate transformed stiffness matrix

Ke = matmul(matmul(transpose(G),Kel),G)

! Print matrix

do i=1,6
    write(*,'(6G10.3)') (Ke(i,j), j=1,6)
end do

end program function_sample
```

Fortran implementation ...

```
0.1925 0.1925 0.1925 -.1925 -.1925 -.1925
0.1925 0.1925 0.1925 -.1925 -.1925 -.1925
0.1925 0.1925 0.1925 -.1925 -.1925 -.1925
-.1925 -.1925 -.1925 0.1925 0.1925 0.1925
-.1925 -.1925 -.1925 0.1925 0.1925 0.1925
-.1925 -.1925 -.1925 0.1925 0.1925 0.1925
```



Program units and subroutines

Building blocks of a structure program

Main program

[**program** program-name]
[specification statements]
[executable statements]
[**contains**
internal-subprograms]
end [**program** [program-name]]

The **end**-statement signals the end of the program-unit and also terminates the program execution

A Fortran program can only have one main program

The stop-statement

- Stops program execution
- Can be used in all program units
- "Well-behaved" subroutines should return control to the main program, where error handling and stop-statements are executed
- A stop-statement can contain stop codes
  - Number or strings
- Bad coding practice in to have stop-codes in subroutines and functions.

Subroutines and functions

- A subroutine/function is a named block of code that performs a certain task
- The subroutine/function can be called from different parts of the program
- A subroutine/function can have a number of arguments that can be used to control the behavior
- Arguments in Fortran can be used to provide both input and output to subroutines/functions
- A function is similar to a subroutine, but can also have a single return value

Subroutines

- External subroutines
  - Placed in separate source files. Implicit interface.
  - Can be implemented in other languages
- Internal subroutines
  - subroutines contained inside program units, external subroutines and in module subroutines
- Module subroutine
  - subroutine contained in a module

Subroutines

A subroutine in Fortran has the following syntax

```
subroutine subroutine-name[([dummy-argument-list])]
    [argument-declaration]
    ...
    return
end subroutine [subroutine-name]
```

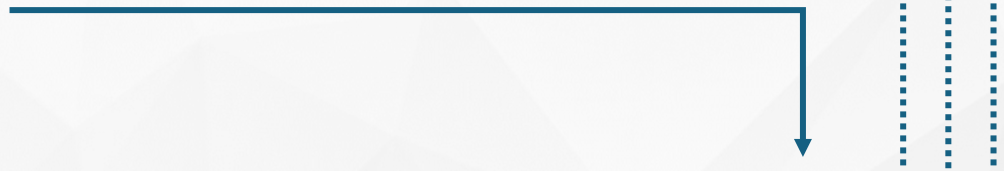
- All variables passed to a subroutine are passed by reference
  - An argument to the subroutine refers to the actual variable in the calling code.
- Subroutine arguments must be declared in the subroutine
- A subroutine is called using the **call** statement
- Control is returned to the calling routine at the end of the subroutine or by calling **return**.
  - Several return statements can exist in a subroutine

Subroutines

```
integer :: a
real, dimension(10,20) :: B
real, dimension(10) :: C

a = 10

call myproc(a,B,C)
```



```
subroutine myproc(r,B,C)

    implicit none

    integer :: r
    real, dimension(r,*) :: B
    real, dimension(r) :: C

    ...

    return

end subroutine
```

Dimensions of arrays need to be passed in the call, so that they can be declared correctly in the subroutine.

Last dimension not needed to be passed (*) as it is not needed to reference an array.

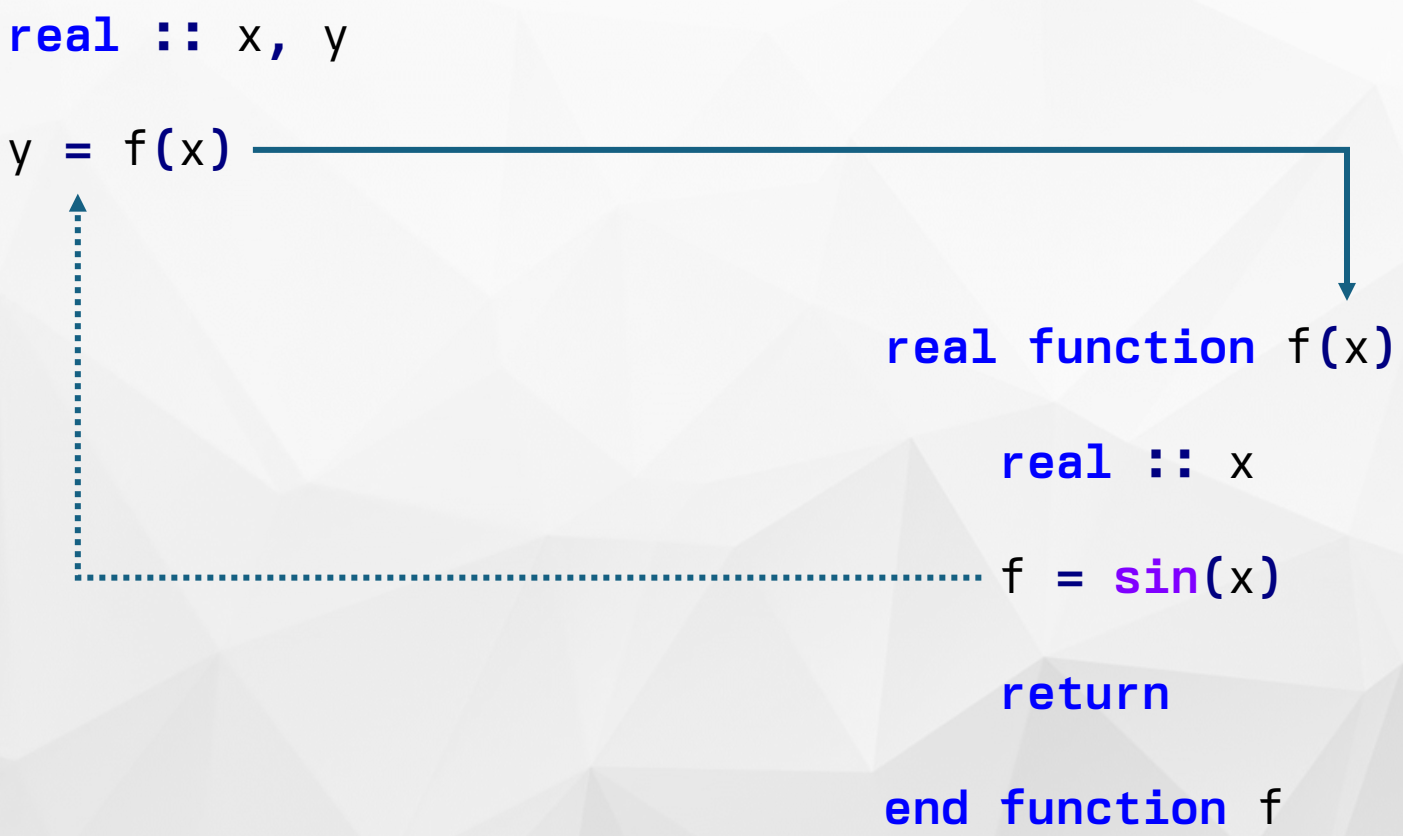
Functions

A subroutine in Fortran has the following syntax

```
type function function-name([dummy-argument-list])
    [argument-declaration]
    ...
    function-name = return-value
    ...
    return
end function function-name
```

"Subroutine with a return value"

Functions


real :: x, y
y = f(x)
real function f(x)
real :: x
f = sin(x)
return
end function f

Modules

- Packages
  - global data
  - derived types and associated operations
  - subprograms
  - interface blocks
  - namelist definitions
- Package everything associated with some kind of task
  - interval arithmetic
  - FEM elements of a certain type ...

Modules

module module-name
    [specification statements]
[**contains**
    module-subprograms]
end [**module** [program-name]]

```
use constants

integer(ik6) :: myint
real(rk) :: myreal
```

```
module constants
    integer, parameter :: ik6 = selected_int_kind(6)
    integer, parameter :: rk = selected_real_kind(15,300)
end module constants
```

Modules

```
module truss

    integer ...
    real, private ...
    public :: bar2e

contains

    subroutine bar2e(...)
       ...
       return
    end subroutine bar2e

end module truss
```



Variables and subroutines can be made private and public (default) using the private and public attributes

Modules

```
program main

    use truss

    call bar2e(...)

end program main
```



All public variables, subroutines and datatypes from truss now available in program unit

Arguments of subroutines

- Paramters passed as reference
  - Changing a parameter in a subroutine affects variable in calling routine
  - Quicker subroutine calls
  - Undesired “sideeffects”
- Declaration of parameters
  - Don’t have to be the same as the calling routine
  - Last dimension can be left out for arrays.

Arguments of subroutines

```
program sub_declaration1

    real(8), dimension(4,4) :: A
    integer, dimension(3) :: v

    A = 0.0_8
    v = 0

    call dowork(A,4,4,v,3)

    print *, A(4,4)
    print *, v(3)

end program sub_declaration1
```

```
subroutine dowork(A,rows,cols,v,elements)

    integer :: rows, cols, elements
    real(8), dimension(rows*cols) :: A
    integer, dimension(elements)  :: v

    A = 42.0_8
    v = 42

    return

end subroutine dowork
```

Subroutine interfaces

- 2 flavors
  - external/implicit interface
  - Internal/explicit interface
- Implicit (Fortran 77)
  - Separately compiled .f90-files
  - Compiler does not have all information
- Explicit (Fortran 9x / 200x)
  - Declared in modules, program contain-section or with interface declaration
  - Compiler has all information on the subroutine interface

Subroutines in Modern Fortran

- Arrays and vectors can be declared with (:,:) or (:) dimensions
- The explicit interface gives the compiler all information to determine the sizes of the parameters
- Shorter subroutine parameter lists
- Actual dimensions of the arrays can be queried using the size(array, dimension) function.

Example

Implicit interface

```
program test

    real(8), dimension(20,30) :: A

    A = 0.0_8

    call mysub(A, 20, 30)

end program test
```

```
subroutine mysub(A,rows,cols)

    integer :: rows, cols
    real(8), dimension(rows,cols) :: A

    A = 42.0_8

    print *, 'rows = ', size(A,1)
    print *, 'cols = ', size(A,2)

    return

end subroutine mysub
```

Example

Module interface

```
program test

    use utils

    real(8), dimension(20,30) :: A

    A = 0.0_8

    call mysub2(A)

end program test
```

```
module utils

contains

subroutine mysub2(A)

    real(8), dimension(:,:) :: A

    A = 42.0_8

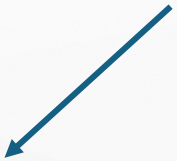
    print *, 'rows = ', size(A,1)
    print *, 'cols = ', size(A,2)

    return

end subroutine mysub2

end module utils
```

Assumed shape array



Internal subroutine

```
program test

    real(8), dimension(20,30) :: A

    A = 0.0_8

    call mysub3(A)

contains

subroutine mysub3(A)

    real(8), dimension(:,:) :: A

    A = 42.0_8

    print *, 'rows = ', size(A,1)
    print *, 'cols = ', size(A,2)

end subroutine mysub3

end program test
```


Assumed shape array

Example

Explicit interface

```
program test

    interface
        subroutine mysub4(A)
            real(8), dimension(:,:) :: A
        end subroutine mysub4
    end interface

    real(8), dimension(20,30) :: A

    A = 0.0_8

    call mysub4(A)

end program test
```

As no module or program unit is used interface must be declared explicitely here

```
subroutine mysub4(A)

    real(8), dimension(:,:) :: A

    A = 42.0_8

    print *, 'rows = ', size(A,1)
    print *, 'cols = ', size(A,2)

    return

end subroutine mysub4
```

Automatic variables

- Temporary variables in subroutines
- Created on the stack
- Automatically allocated to desired size
- Must fit in the stack

Example

Automatic variables

```
program automatic_arrays

    real(8), dimension(20,30) :: A

    call mysub(A)

contains

subroutine mysub(A)

    real(8), dimension(:,:) :: A
    integer, dimension(size(A,1),size(A,2)) :: B

    B = 0

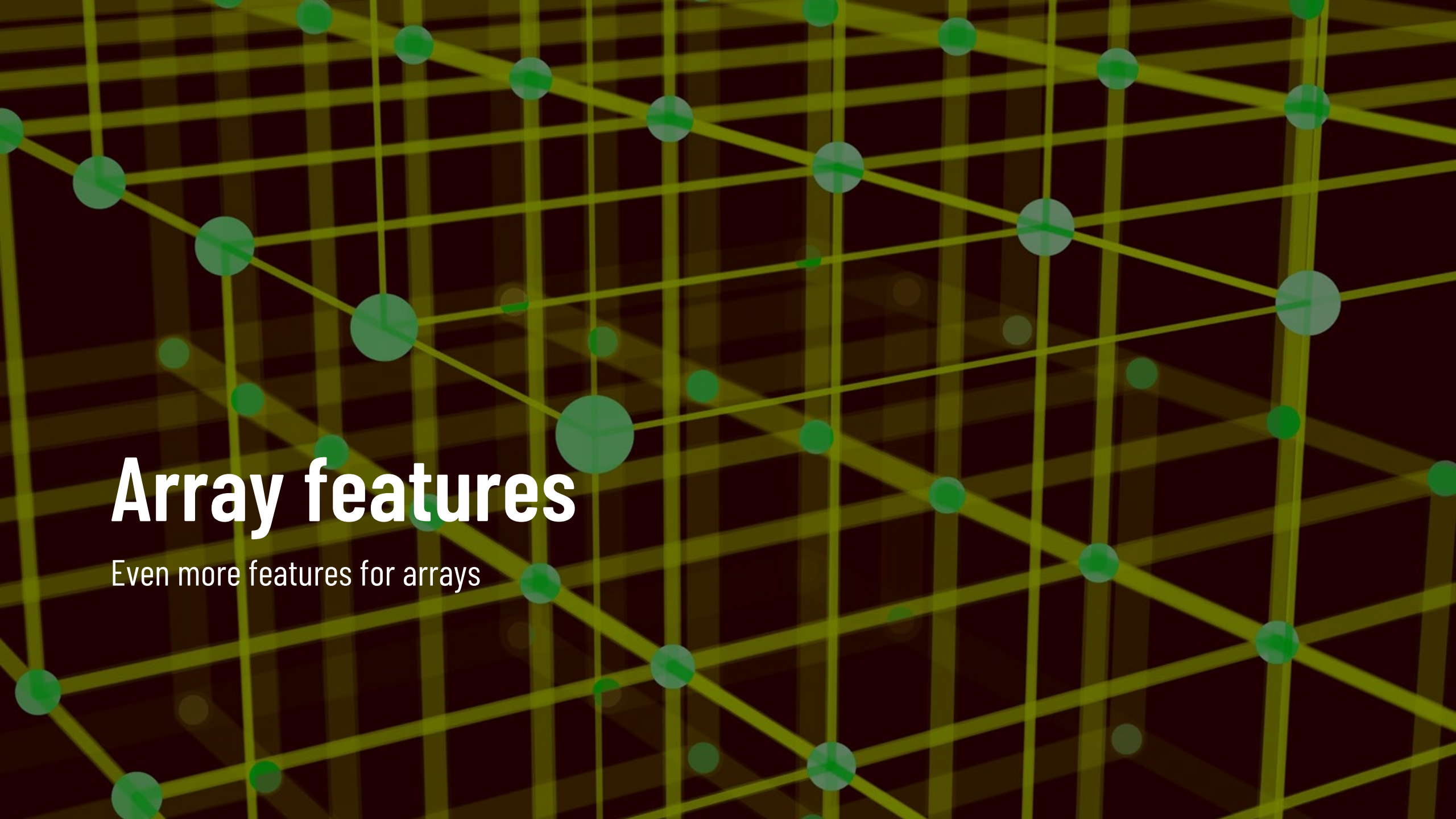
    print *, "rows = ", size(B,1)
    print *, "cols = ", size(B,2)

end subroutine mysub

end program automatic_arrays
```



B is automatically allocated. size() can be used to query size. More on this later.



Array features

Even more features for arrays

Allocatable arrays

- Size of arrays not always known at compile time
- Declared using the **allocatable** attribute
- The array rank is defined at declarations, but bounds undefined until array has been allocated
- **allocate**-statements used to allocate the array
- "Work arrays" where used in Fortran 77 as a replacement for allocatable arrays
- Deallocated with **deallocate**-statement
- Allocatable arrays automatically deallocated when returning from a function or subroutine

Allocatable arrays

```
real, dimension(:), allocatable :: f
real, dimension(:,:), allocatable :: K

allocate(f(20))      ! or allocate(f(20), K(20,20)
allocate(K(20,20))

deallocate(f)        ! or deallocate(f, K)
deallocate(K)
```

Array subobjects

- Used to extract fields/arrays from other arrays
- Can be used as parameters in subroutine calls
- Compare Matlab's index notation

Array subobjects

```
program array_subobjects

    integer, dimension(5,10) :: A
    integer :: i, j

    do i=1,5
        do j=1,10
            A(i,j) = (i-1)*10 + j
        end do
    end do

    print *, "Entire array:"
    call writeArray(A)

    print *, "A(1, 5:10)"
    call writeVector(A(1, 5:10))

    print *, "A(1:5, 5:10)"
    call writeArray(A(1:5, 5:10))

    print *, "A(1:5, 5:10)"
    call writeArray(A(1:5, 5:10))

    print *, "A((/1,3/), (/2,4/))"
    call writeArray(A( (/1,3/), (/2,4/) ))

end program array_subobjects
```

```
Entire array:
1 2 3 4 5 6 7 8 9 10
11 12 13 14 15 16 17 18 19 20
21 22 23 24 25 26 27 28 29 30
31 32 33 34 35 36 37 38 39 40
41 42 43 44 45 46 47 48 49 50
A(1, 5:10)
5 6 7 8 9 10
A(1:5, 5:10)
5 6 7 8 9 10
15 16 17 18 19 20
25 26 27 28 29 30
35 36 37 38 39 40
45 46 47 48 49 50
A(1:5, 5:10)
5 6 7 8 9 10
15 16 17 18 19 20
25 26 27 28 29 30
35 36 37 38 39 40
45 46 47 48 49 50
A((/1,3/), (/2,4/))
2 4
22 24
```

Vector and matrix multiplication

- Matrix multiplication
  - **C = matmul(A,B)**, where **A**, **B** and **C** are matrices
- Scalarproduct
  - **c = dot_product(a,b)**, where **a** and **b** are vectors and **c** a scalar

Matrix multiply

```
program matrix_multiply

    real(8), dimension(4,4) :: A
    real(8), dimension(4,8) :: B
    real(8), dimension(4,8) :: C

    call randomMatrix(A)
    call randomMatrix(B)
    call randomMatrix(C)

    C = matmul(A,B)

    print *, "A = "
    call writeArray(A)
    print *, "B = "
    call writeArray(B)
    print *, "C = "
    call writeArray(C)

end program matrix_multiply
```

```
 A =
.984    .700    .275    .661
.810    .910    .304    .484
.985    .347    .548    .614
.971    .882    .129    .929
 B =
.763    .534    .711E-01.404    .731    .636    .879    .689
.786    .231    .238    .951    .621    .528    .733    .588E-01
.466    .302    .209    .294    .598    .461    .911    .343
.690E-01.619    .305    .326E-01.438    .202    .617    .851
 C =
1.48    1.18    .496    1.17    1.61    1.26    2.04    1.38
1.51    1.03    .486    1.30    1.55    1.23    1.96    1.13
1.32    1.15    .455    .909    1.53    1.19    2.00    1.41
1.56    1.34    .589    1.30    1.74    1.33    2.19    1.56
```

Reduction routines

- **all(...)**
  - True if all element = **.true.**
- **any(...)**
  - True if any element = **.true.**
- **maxval(...)/minval(...)**
  - Max/min value in the field
- **product(...)**
  - Product of all elements in the field
- **sum(...)**
  - Sum of all elements in the field

Reduction routines

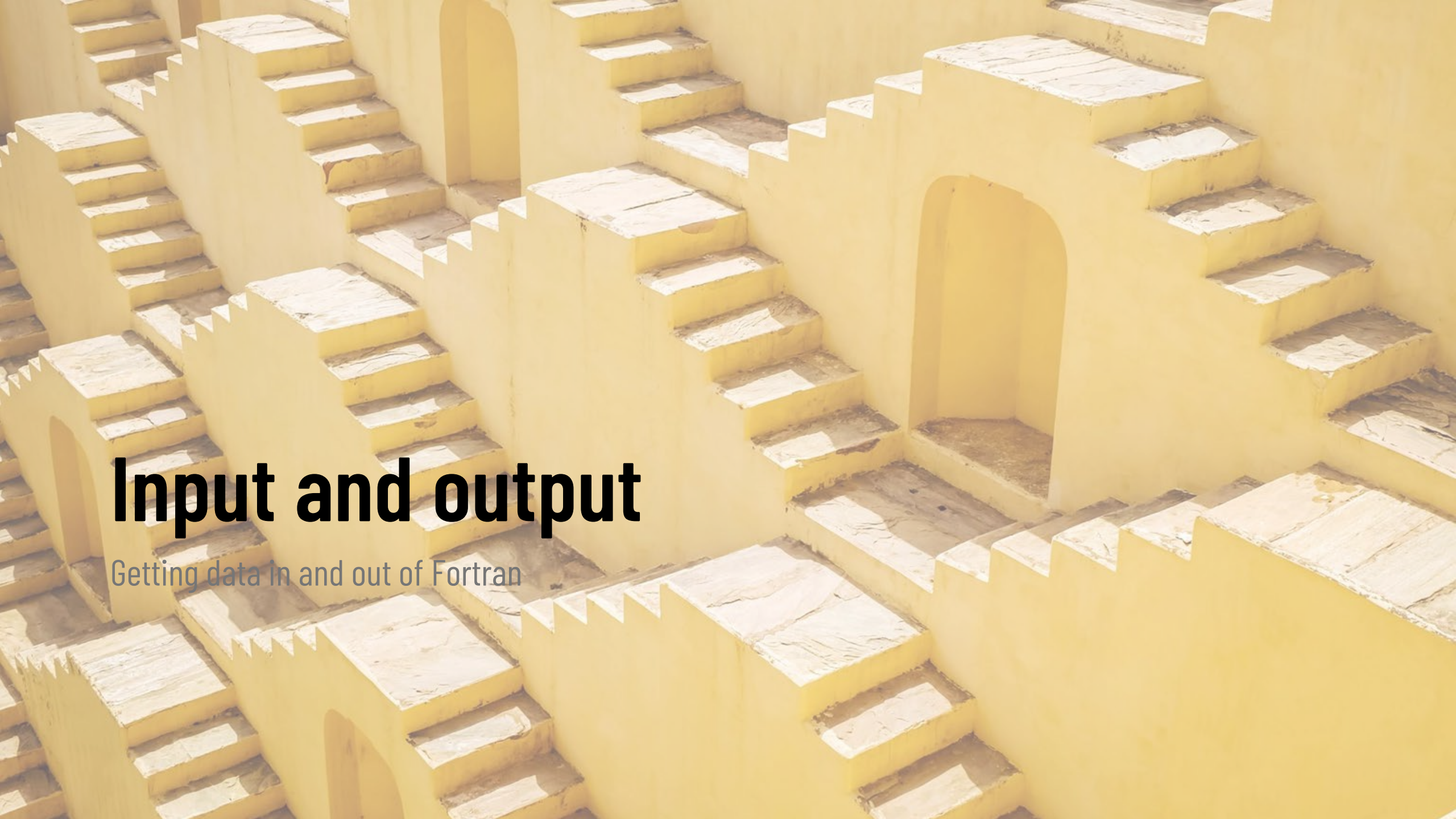
```
print *, 'Product of A = ', product(A)
print *, 'Sum of A     = ', sum(A)
print *, 'Max of A     = ', maxval(A)
print *, 'Min of A     = ', minval(A)
```

```
 A =
.984     .700     .275     .661
.810     .910     .304     .484
.985     .347     .548     .614
.971     .882     .129     .929

Product of A =  0.00016095765496870103
Sum of A     =  10.534961942117661
Max of A     =  0.9853920286986977
Min of A     =  0.12899155798368156
```

Information routines

- **lbound(...)**
  - Returns lower field index
- **ubound(...)**
  - Return upper field index
- **shape(...)**
  - Returns the matrix shape in an array [rows, columns]
- **size(...)**
  - Returns the size of matrix in a specific dimension



Input and output

Getting data in and out of Fortran

Reading and writing

- Reading/writing from files is done using the **read/write**-statements
- One row for each **read/write**-statement
- First parameter determines where to read and write.
  - When reading from standard input or writing to standard output an asterisk (*) kan be used as the first parameter
- Fortran automatically converts text in a file to typed variable values
- Several variables can be read in the same statement using a comma separated list of variables
- To handle a large number of values on a row an implicit loop can be used

Reading and writing

```
! Read 8 floating point values from unit ir

read(*,*) a(1), a(2), a(3), a(4), a(5), a(6), a(7), a(8)

! or perhaps even better...

read(*,*) (a(i), i=1,8)

! Same procedure for writing

write(*,*) (a(i), i=1,8)
```

For only writing to standard output, the print-statement can be used.

```
print*, 'Hello, world!'
```

Formatted I/O

- Second parameter in the **write**-statement defines the output format
- * represents the default format which often displays at full precision
- A format parameter is a string describing the output format
- The format is defined using format codes

Formatted I/O

- Common codes
  - **A[n]** – character string with the width n
  - **I[n]** – integer with the width n
  - **G[m.n]** – floating point value with width m and n decimals
- Format codes can be repeated by giving a integer in front of the code
  - **'(6I5)'** = 6 integers with the width 5

Example

Formatted I/O

```
program format_print

    real(8), dimension(3) :: a
    integer, dimension(4) :: b
    integer :: i, j

    ! Write table header

    a = (/ 1.0, 2.0, 3.0 /)
    b = (/ 4, 5, 6, 7 /)

    write(*, '(3A8,4A5)') 'aaaaaaaa', &
        'bbbbbbbb', 'cccccccc', 'ddddd', 'eeeee', 'fffff', &
        'ggggg‘

    ! Write table

    do j=1,10
        write(*, '(3G8.3,4I5)') (a(i),i=1,3), (b(i), i=1,4)
    end do

end program format_print
```

Formatted I/O

```
aaaaaaaabbbbbbbbccccccccddddddeeeeeffffffgggggg
1.00     2.00     3.00          4     5     6     7
1.00     2.00     3.00          4     5     6     7
1.00     2.00     3.00          4     5     6     7
1.00     2.00     3.00          4     5     6     7
1.00     2.00     3.00          4     5     6     7
1.00     2.00     3.00          4     5     6     7
1.00     2.00     3.00          4     5     6     7
1.00     2.00     3.00          4     5     6     7
1.00     2.00     3.00          4     5     6     7
1.00     2.00     3.00          4     5     6     7
1.00     2.00     3.00          4     5     6     7
```

Dynamic format codes

- Format strings are static strings
- How to create a format code that can handle dynamic number of values
- **write(...)** can use a character string as a “file”

Example

Dynamic format codes

```
subroutine writeArray(A)

    real(8), dimension(:,:) :: A
    integer :: rows, cols, i, j
    character(255) :: fmt

    rows = size(A,1)
    cols = size(A,2)

    write(fmt, '(A,I1,A)') '(',cols, 'G8.3)'

    do i=1,rows
        print fmt, (A(i,j), j=1,cols)
    end do

    return

end subroutine writeArray
```

String that will contain the final format code.

Write to fmt string using a write statement

Use the format string when printing array

Reading and writing to files

- To read and write data to file a file must be associated with a file unit (integer number)
  - The **open()**-statement is used to accomplish this
  - Also determines if the file should be read, written or appended.
- There are 3 modes of file operations in Fortran
  - Text I/O - Textfiles
  - Namelist I/O - Textfiles with named variables
  - Unformatted I/O - Binary files
- When file I/O is completed files needs to be closed
  - Tells the operating environment that a file is not used
  - If files are not closed there is a risk that they won't be written correctly to disk (due to buffering)

Opening files

```
integer, parameter :: infile = 15
integer, parameter :: outfile = 16

! ---- Open a file for reading

open(unit=infile,file='indata.dat', access='sequential', action='read')

! ---- Read from file

read(infile, *) ...

! ---- Close file

close(infile)

! ---- Open a file for writing

open(unit=outfile,file='utdata.dat', access='sequential', action='write')

! ---- Write to file

write(outfile, *) ...

! ---- Close file

close(outfile)
```

Texfile mode

File operation

Filename

Reading from file and keyboard

```
read(*,*) A
read(5,*) A
```

Keyboard

```
integer, parameter :: ir=15

open(unit=ir, file='example.dat')
read(ir,*) A
close(unit=ir)
```

File

Writing to file and screen

```
write(*,*) A
write(6,*) A
write(*,' (TR1,A,G15.6)') 'A = ', A
```

Screen

```
integer, parameter :: iw=16

open(unit=iw, file='example.dat')
write(iw, '(A,G15.6)') A
close(unit=iw)
```

File



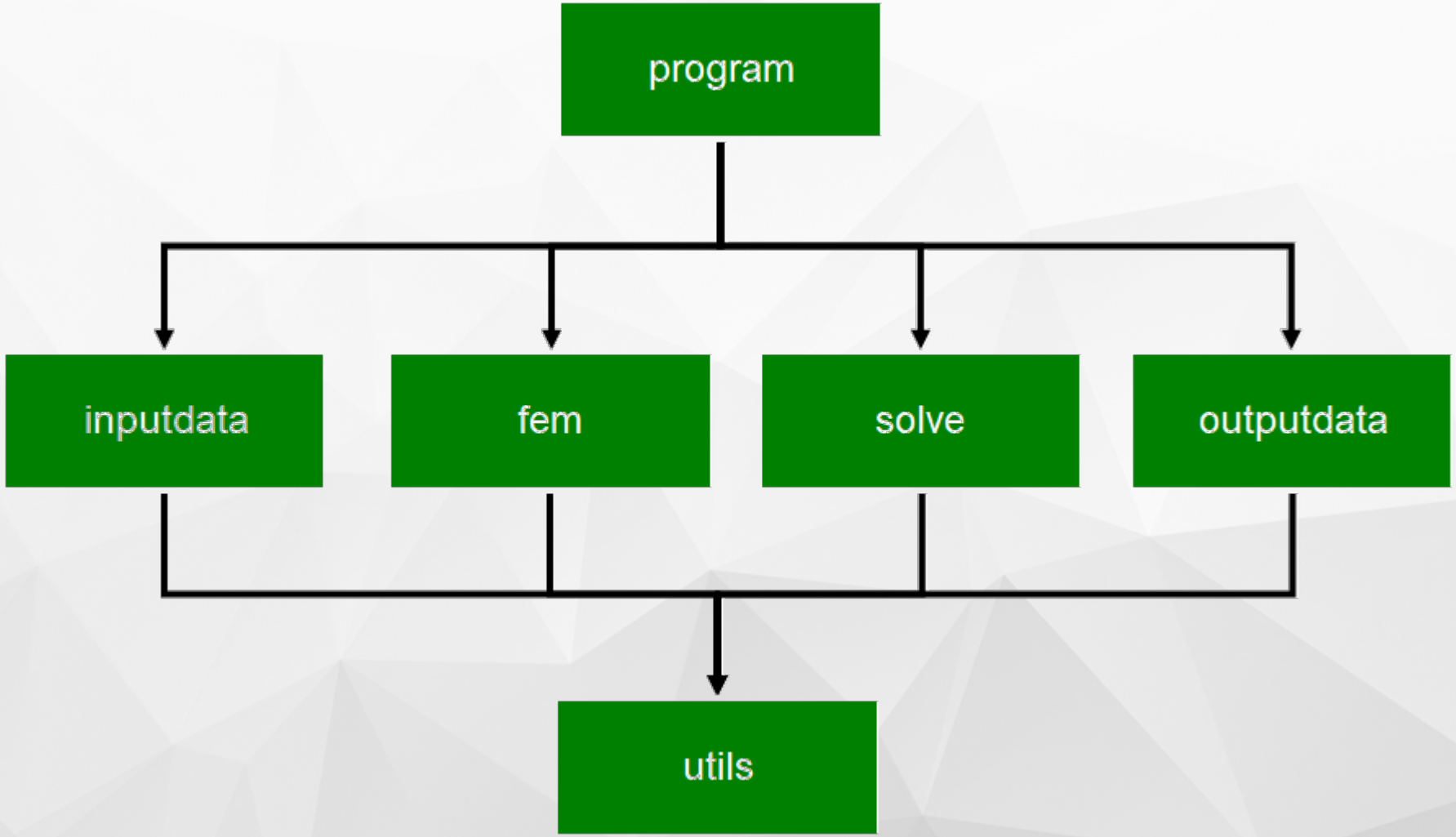
Code structuring

Building larger applications in Fortran

Structuring of projects

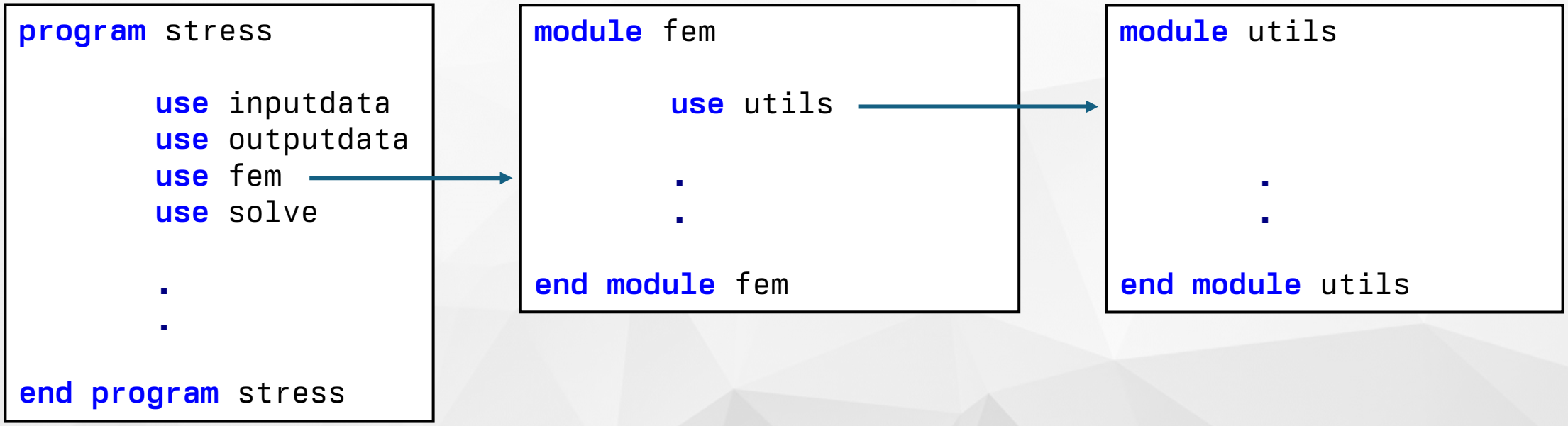
- For smaller project modularisation not necessary
- For larger project it is a must to be able to maintain the code
- In Modern Fortran the key to modularisation is the module
- Modules should be used to group related subroutines and data

Structuring of projects



Qt Creator: fea_app

Structuring of projects





More on subroutines

Subroutine features of Modern Fortran

Procedures as arguments

- procedures in Modern Fortran can take procedures as arguments
- Properties of “dummy” procedure arguments must agree
- The procedure type is declared in the specification part
- Internal procedures can’t be used

Procedures as arguments

```
module utils

            implicit none

        integer, parameter :: dp = selected_real_kind(15,300)

contains

real(dp) function myfunc(x)

            real(dp), intent(in) :: x

            myfunc = sin(x)

end function myfunc

subroutine tabulate(startInterval, endInterval, step, func)

            real(dp), intent(in) :: startInterval, endInterval, step
            real(dp) :: x

            interface
                        real(8) function func(x)
                                    real(8), intent(in) :: x
                        end function func
            end interface

            x = startInterval
            do while (x<endInterval)
                        print *, x, func(x)
                        x = x + step
            end do

            return
end subroutine tabulate

end module utils
```

Declaration of function interface. Function used as argument must have the same interface

Procedures as arguments

```
program procedures_as_arguments

      use utils

      implicit none

      call tabulate(0.0_dp, 3.14_dp, 0.1_dp, myfunc)

end program procedures_as_arguments
```

Procedures as arguments

```
 0.0000000000000000       0.0000000000000000
 0.10000000000000001      9.98334166468281548E-002
 0.20000000000000001      0.19866933079506122
 0.30000000000000004      0.29552020666133960
 0.40000000000000002      0.38941834230865052
 0.50000000000000000      0.47942553860420301
 0.59999999999999998      0.56464247339503537
 0.69999999999999996      0.64421768723769102
 0.79999999999999993      0.71735609089952268
 0.89999999999999991      0.78332690962748330
 0.99999999999999989      0.84147098480789639
  1.0999999999999999      0.89120736006143531
  1.2000000000000000      0.93203908596722629
  1.3000000000000000      0.96355818541719296
  1.4000000000000001      0.98544972998846025
  1.5000000000000002      0.99749498660405445
  1.6000000000000003      0.99957360304150511
  1.7000000000000004      0.99166481045246857
  1.8000000000000005      0.97384763087819504
  1.9000000000000006      0.94630008768741425
  2.0000000000000004      0.90929742682568149
  2.1000000000000005      0.86320936664887349
  2.2000000000000006      0.80849640381958987
  2.3000000000000007      0.74570521217671970
  2.4000000000000008      0.67546318055115029
  2.5000000000000009      0.59847214410395577
  2.6000000000000010      0.51550137182146338
  2.7000000000000011      0.42737988023382895
  ...
```

Keyword and optional arguments

- Many procedures have long argument lists
  - All arguments not needed
- Procedural arguments in Fortran can be given the attribute optional
  - **real**, **optional** :: **a**
- Optional arguments can be omitted in the procedure call
  - Presence of an argument can be tested with the present function.
- Arguments can be specified with keywords

Keyword and optional arguments

```
call order_icecream(2)
call order_icecream(2, 1)
call order_icecream(4, 4, 2)
call order_icecream(4, topping=3)
```

```
subroutine order_icecream(number, flavor, topping)

        integer :: number
        integer, optional :: flavor
        integer, optional :: topping

        print *, number, 'icecreams ordered.'

        if (present(flavor)) then
                print *, 'Flavor is ', flavor
        else
                print *, 'No flavor was given.'
        end if

        if (present(topping)) then
                print *, 'Topping is ', topping
        else
                print *, 'No topping was given.'
        end if

end subroutine order_icecream
```

Overloading

- Procedure call to different procedures depending on datatype
- Declare an interface with procedural interfaces for all used datatypes
- Explicit interfaces (**modules**) can use a simpler form
  - **module procedure** procedure-name-list

Overloading

```
1764
1.0000000000000000
```

```
program overloading

        use special

        implicit none

        integer :: a = 42
        real(dp) :: b = 42.0_dp

        a = func(a)
        b = func(b)

        print *, a
        print *, b

end program overloading
```

```
module special

interface func
        module procedure ifunc, rfunc
end interface

contains

integer function ifunc(x)

        integer, intent(in) :: x
        ifunc = x * 42

end function ifunc

real(dp) function rfunc(x)

        real(dp), intent(in) :: x
        rfunc = x / 42.0_dp

end function rfunc

end module special
```



Operator overloading

- Define operations on derived types
- Vector / Matrix operations
- **interface** operator(operator) used to define a function implementing the operations for this operator
- Function has the form
  - **type**(derived-type-name) function-name(operand1, operand2)

Operator overloading

```
module vector_operations

        integer, parameter :: dp = selected_real_kind(15,300)

         type vector
                 real(dp) :: components(3)
         end type vector

         interface operator(+)
                 module procedure vector_plus_vector
         end interface

contains

type(vector) function vector_plus_vector(v1, v2)

         type(vector), intent(in) :: v1, v2
         vector_plus_vector%components = v1 % components + v2 % components

end function vector_plus_vector

end module vector_operations
```

Operator overloading

```
program operator_overloading

    use vector_operations

    implicit none

    type(vector) :: v1
    type(vector) :: v2
    type(vector) :: v

    v1 % components = (/1.0, 0.0, 0.0/)
    v2 %components = (/0.0, 1.0, 0.0/)

    v = v1 + v2

    print *, v

end program operator_overloading
```

```
1.0000000000000000   1.0000000000000000   0.0000000000000000
```

Public and private properties

- Control visibility of variables and functions
- Hide module implementation details
- Prevent side effects in modules
- Visibility of variables controlled by **private** and **public** attributes
- procedure visibility controlled by
  - **private** access-list
- **public** is the default for all module variables and procedures

Public and private attributes

```
module mymodule

    implicit none

    integer, public :: visible
    integer, private :: invisible

    private privatefunc
    public publicfunc

contains

subroutine privatefunc

    print *, 'This function can only be called from within a module.'

end subroutine privatefunc

subroutine publicfunc

    call privatefunc

end subroutine publicfunc

end module mymodule
```

Public and private attributes

```
program private_entities

    use mymodule

    implicit none

    call publicfunc

end program private_entities
```

Works as **publicfunc** is public

```
program private_entities

    use mymodule

    implicit none

    call privatefunc

end program private_entities
```



```
make all
gfortran -g -fbounds-check -Wall -Wtabs -c main.f90
gfortran  -o private_entities mymodule.o main.o
main.o: In function `private_entities':
D:\edu\.../main.f90:7: undefined reference to `privatefunc_'
collect2: ld returned 1 exit status
make: *** [private_entities] Error 1
```

Doesn't work as privatefunc is private


Pointers
Pointing at data in memory

Pointers

- Pointer = reference to a variable
- Strictly typed
- Pointer targets must have **target** attribute
  - For compiler optimisation
- Enables the return of allocatable arrays from procedures
- **nullify(pointer)** disassociates pointer from target
- => operator is used to associate a pointer with a target

Pointers

Qt Creator: pointers

```
program pointers

    implicit none

    integer, allocatable, dimension(:,:), target :: A
    integer, dimension(:,:), pointer :: B, C

    allocate(A(20,20))

    B => A

    print *, size(B,1), size(B,2)

    call createArray(C)

    print *, size(C,1), size(C,2)

    deallocate(C)

    B => null()

    B(1,1) = 0 ! Dangerous!

contains

subroutine createArray(D)

    integer, dimension(:,:), pointer :: D

    allocate(D(10,10))

end subroutine createArray

end program pointers
```

B points to A

B can be queried just like a normal array

An unassociated pointer can be passed to a subroutine and be returned as an allocated array



Pointer usage

- Pointers can be used to create more complex data structures such as
  - Linked lists
  - Hash maps
  - Binary trees
  - ...
- Pointers can also be contained in derived data types



Advanced I/O

More ways of data management in Fortran

List directed I/O

- Fortran uses a specific format for reading and writing variables to files
- Variables output from write(*,*) are separated with space/blanks
  - Complex numbers surrounded by ( )
  - Strings must be enclosed in ‘ ‘ or “ “ when output
- Reading variables require the same format

List directed I/O

```
program list_io

    implicit none

    integer, parameter :: ir = 15
    integer, parameter :: iw = 16

    integer :: a = 42
    real :: b = 42.0 * 42.0
    character(len=20) :: string = 'Hello, string!'
    logical :: topping = .true.
    complex :: c = (1.0,2.0)

    open(unit=iw, file='list.txt', status='replace')
    write(iw,*) a, b, '"',string,'"', topping, c
    close(unit=iw)

    open(unit=ir, file='list.txt', status='old')
    read(ir,*) a, b, string, topping, c
    close(unit=ir)

    print *, a
    print *, b
    print *, topping
    print *, c

end program list_io
```



Namelist I/O

- Reading and writing list of named values
- Value names defined in a namelist
- Input must consist of lists of named value pairs separated with comma
  - & name = value, name = value /
- Can contain arrays
  - name = 1,2,3,4,5

Namelist I/O

```
program namelist_io

    implicit none

    integer, parameter :: ir = 15
    integer, parameter :: iw = 16
    integer :: no_of_eggs, litres_of_milk, kilos_of_butter, list(5)
    namelist /food/ no_of_eggs, litres_of_milk, kilos_of_butter, list

    list = 0

    open(unit=ir, file='food.txt', status='old')
    read(ir, nml=food)
    close(unit=ir)

    print *, no_of_eggs, litres_of_milk, kilos_of_butter

    open(unit=iw, file='food2.txt', status='new')
    write(iw, nml=food)
    close(unit=iw)

end program namelist_io
```

namelist defined variables to be read from file





namelist defined variables to be written to file.

Unformatted I/O (binary files)

- Writing data using a textual representation not efficient in all cases
- No format needed, binary output
  - read(unit=xx) a
  - write(unit=xx) b
- Output from unformatted I/O often system dependent
- Scratchfiles and temporary files

Direct access files

- Used when a random access pattern is needed
- File consists of a set of equal sized records
- Any record in the file can be accessed randomly
- If the record is changed data cannot be read back from file
- Size of a record can be found using **inquire(iolength=**variable-name**)** recordLength

Direct access files / unformatted I/O

```
program unformatted_io

implicit none

    type account
        character(len=40) :: account_holder
        real :: balance
    end type account

    integer, parameter :: iw = 15
    type(account) :: accountA
    type(account) :: accountB
    integer :: recordSize

    inquire(iolength=recordSize) accountA

    print *, 'Record size =',recordSize

    accountA % account_holder = 'Olle'
    accountA % balance = 400

    accountB % account_holder = 'Janne'
    accountB % balance = 800

    open(unit=iw, file='bank.dat', access='direct', recl=recordSize, status='replace')
    write(iw, rec=1) accountA
    write(iw, rec=2) accountB
    close(unit=iw)

end program unformatted_io
```

Derived datatypes good candidates for storing as records in a random access file

Inquire for the required record length

access=’direct’ creates a direct access file.

The rec attribute determines the location to store/read the record

File positioning

- File pointer – Invisible cursor in the file
- **backspace** unit
  - Move file pointer back 1 record
- **rewind** unit
  - Move file pointer to beginning of file
- **endfile** unit
  - Move file pointer to the end of the file.

Error handling in file operations

- **open(...)**, **close(...)**, **read(...)** and **write(...)** have an err attribute that can be set to a specific label to jump to when an error occurs
- Well behaved programs should implement error handling when reading and writing files

Error handling in file operations

```
program error_handling

    implicit none

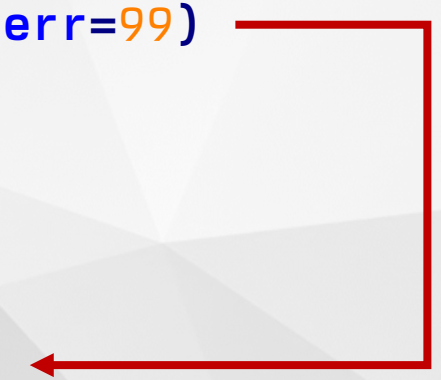
    integer, parameter :: ir = 15
    integer :: a

    open(unit=ir, file='test.txt', status='old', err=99)
    read(ir,*,err=99) a
    close(unit=ir,err=99)

    stop

99  print *, 'An error occured reading the file.'

end program error_handling
```





Modern Fortran features

Fortran has many new tricks

Allocatable arrays as dummy arguments

- Remove the need for pointers to use allocatable arrays as dummy arguments
- Inefficient

Allocatable arrays as dummy arguments

Example

```
program allocatable_dummy
    implicit none
    real, allocatable :: A(:,:)
    call createArray(A)
    print *, size(A,1), size(A,2)
    deallocate(A)
contains
subroutine createArray(A)
    real, allocatable, intent(out) :: A(:,:)
    allocate(A(20,20))
end subroutine createArray
end program allocatable_dummy
```



A pointer dummy argument was needed to implement this i Fortran 95

Allocatable arrays as return values

Example

```
program allocatable_function

    implicit none

    real :: A(20)

    A = createVector(20)
    print *, size(A,1)

contains

function createVector(n)

    real, allocatable, dimension(:) :: createVector
    integer, intent(in) :: n

    allocate(createVector(n))

end function createVector

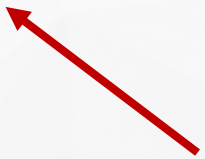
end program allocatable_function
```



Vector allocated in function will be automatically deallocated when it has been used.

Allocatable arrays in derived data types

```
type stack
    integer :: index
    integer, allocatable :: content(:)
end type stack
```



Pointers used in Fortran 95, but no automatic deallocation available

Submodules

- The Fortran 95 module concept adequate for moderate size projects
- Difficult to modularize modules themselves
- Dividing modules into several modules
  - exposes internal structure
  - nameclashes
- Changes to module features require recompilation of all files using this module
- Separation of module into
  - Interface – defined in the module
  - Body – defined in submodule

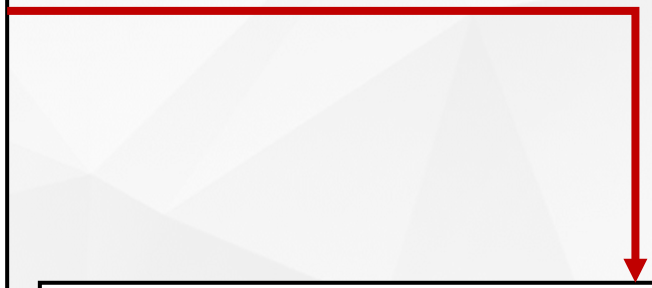
Submodules

```
module points

type point
    real :: x, y
end type point

interface
    real module function point_dist(a, b)
        type(point), intent(in) :: a, b
    end function point_dist
end interface

end module points
```



```
submodule (points) points_a

contains

real module function point_dist(a,b)
    type(point), intent(in) :: a, b
    point_dist = sqrt((a%x-b%x)**2+(a%y-b%y)**2)
end function point_dist

end submodule points_a
```

C interoperability

- No clear definitions existed between Fortran 95 and the C language
- C/Fortran programming is important
  - Interfacing with C libraries
  - C user interface application interfacing with Fortran code
- New intrinsic module, **iso_c_binding**, included
  - Contains definitions of all the C datatypes mapped to Fortran types

Example

C interoperability

```
program c_interop

    use iso_c_binding

    implicit none

    integer(c_int) :: a
    real(c_float) :: b
    real(c_double) :: c

    a = 42
    b = 42.0_c_float
    c = 84.0_c_double

    print *, a, b, c

end program c_interop
```

C interoperability

- Fortran always passes all variables by reference
- In C scalar arguments are often passed by value
  - Argument value is copied to procedure
- The variable attribute value is added for compatibility with C code
- Enables direct linking with C

Access to computing environment

- **get_environment_variable(...)**
  - Query environment variables
- **command_argument_count()**
  - Retrieve the number of command line arguments
- **get_command_argument(...)**
  - Retrieve specific command line argument



Tools for Modern Fortran Development

Tools that make Fortran development easy and efficient

Maintaining Fortran code

- Compilation and maintainance issues grow as project becomes larger
- Requirements for building on multiple platforms
- Shell scripts not suitable for larger projects
  - Can’t handle source dependencies
  - Require complete rebuild for every change
- Make and CMake can solve some of these problems

Maintaining Fortran projects with Make

Make

- Builds software from using a set of rules
- Automatically handles dependencies in source files
  - Only rebuilds files affected by the change in the source files
- Available on all platforms (GNU Make)
- Can be non-intuitive in the beginning

Basic make file syntax

```
target: [dependencies]
→ system command
```

TAB

This is the target of the rule

Myfile.gz depends on myfile.txt

```
myfile.gz: myfile.txt
→ cat myfile.txt | gzip > myfile.gz
```

TAB

This is the "recipe" for creating myfile.gz

Running make

```
$ ls
Makefile myfile.txt
$ make
cat myfile.txt | gzip > myfile.gz
$ ls
Makefile myfile.gz myfile.txt
```

Running make again

```
$ make
make: ’myfile.gz’ is up to date.
```

Running make with updated myfile.txt

```
$ touch myfile.txt
$ make
cat myfile.txt | gzip > myfile.gz $
```

Compiling code

- Building an executable requires
  - Compilation of source code to object-files
  - Linking of object-files to executable
- Object-files depend on source files
- Executable depends on object-files
- Case for make ;)

Example

```
myprog : myprog.o
	gfortran myprog.o -o myprog

myprog.o : myprog.f90
	gfortran -c myprog.f90
```

Link executable with object-files

Compile source code to object file

Running make

```
$ ls
Makefile myprog.f90
$ make
gfortran -c myprog.f90
gfortran myprog.o -o myprog
```

Example 2

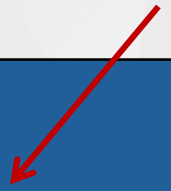
```
myprog : mymodule.o myprog.o
	gfortran mymodule.o myprog.o -o myprog

myprog.o : myprog.f90
	gfortran -c myprog.f90

mymodule.o : mymodule.f90
	gfortran -c mymodule.f90
```

Running make

No compilation of myprog.o



```
$ touch mymodule.f90
$ make
gfortran -c mymodule.f90
gfortran myprog.o mymodule.o -o myprog
```

Module dependencies

- In Fortran 9x, compilation order is important
- A module used by another source file needs to be built earlier in the build
- Make can solve this for us again ;)

Example 3

```
myprog : module main.o module_truss.o
        gfortran module main.o module_truss.o -o myprog

module_main.o : module_main.f90
        gfortran -c module_main.f90

module_truss.o: module_truss.f90
        gfortran -c module_truss . f90
```

Running make

```
$ make
module_main.f90:3.5:

use truss
        1

Fatal Error: Can't open module file 'truss.mod' for reading at (1):
No such file or directory
make: *** [module_main.o] Error 1
```



Module_truss needs to built before module_main

Example 3 - Modified

```
myprog : module main.o module_truss.o
	gfortran module main.o module_truss.o -o myprog

module_main.o : module_main.f90 module_truss.o
	gfortran -c module_main.f90

module_truss.o: module_truss.f90
	gfortran -c module_truss . f90
```

Module_truss is now build before module_main

Running make

```
$ make
gfortran -c module_truss.f90
gfortran -c module_main.f90
gfortran module_main.o module_truss.o -o myprog
```

Variables in make

- Reduce rewrite of common commands
- Increase configuratbility and portability
- Easier to read
- Works in principle as bash variables
- Assignment
  - [Variablename] = value
- Using value of variable
  - $([Variablename])

Example 4

```
FC=gfortran
FFLAGS=-c
EXECUTABLE=myprog

$(EXECUTABLE) : myprog.o mymodule.o
	$(FC) myprog.o mymodule.o -o myprog

Myprog.o : myprog.f90
	$(FC) $(FFLAGS) myprog.f90

Mymodule.o : mymodule.f90
	$(FC) $(FFLAGS) mymodule.f90

clean :
	rm -rf *.o *.mod $(EXECUTABLE)
```

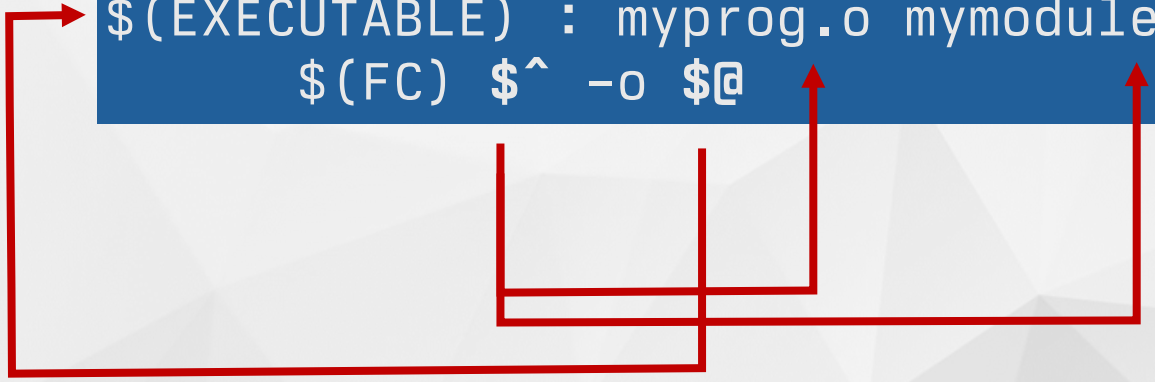
All configurable options located in a single position in the makefile

Internal macros

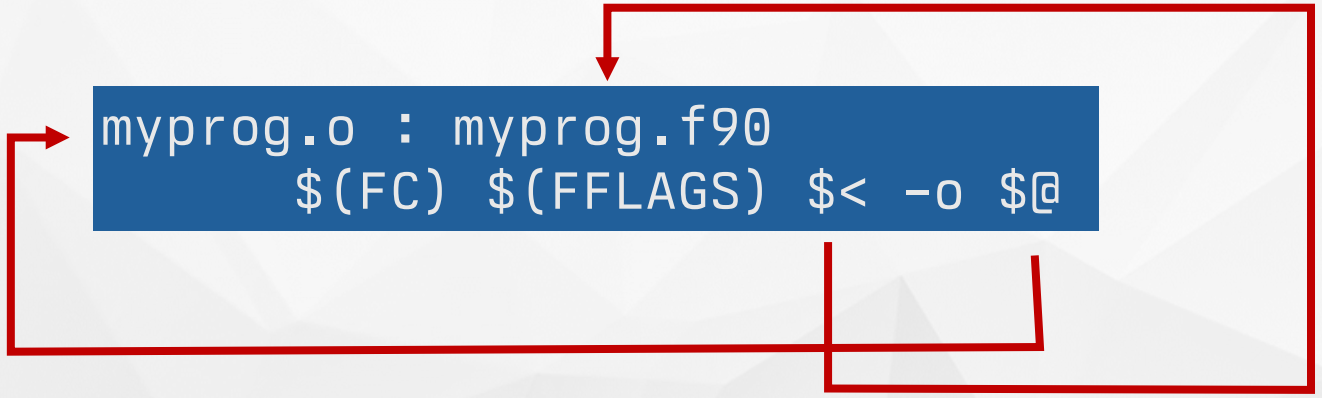
- Create even more generic makefiles
- Most important macros
  - **$@** - Target of the current rule executed
  - **$^** - Name of all pre-requisites
  - **$<** - Name first pre-requisite

Example 5

```
FC=gfortran
FFLAGS=-c
EXECUTABLE=myprog
$(EXECUTABLE) : myprog.o mymodule.o
	$(FC) $^ -o $@
```



Example 5 – cont…



Suffix rules

- In larger projects it can be difficult to maintin a large number of rules for single source files
- Suffix rules are “recipies” for converting a source suffix to an object suffix
  - Same rule applies to all source code in makefile

Example 6 – cont...

If an executable depends on **main.o** this rule will automatically try to build it using **main.f90**

```
.f90.o:
	$(FC) $(FFLAGS) $< -o $@

.SUFFIXES: .f90 .f03 .f .F
```

A suffix must be supported by make. Additional suffices can be added using the **.SUFFIXES** rule.

Wildcard expansion and more

- Wildcard expansion using the wildcard-function can be used to create lists of files
- Substitutions can also be made using the patsubst-function in make
- This enables even more generic makefiles

Example 7

```
F90_FILES := $(wildcard *.f90)
```

Finds all .f90 files in the current directory and assigns them to the list F90_FILES.

NOTE, the **:=** assignment operator must be used in make-functions

```
OBJECTS := $(patsubst %.f90, %.o, $(F90_FILES))
```

Replaces all **.f90** suffixes with **.o** assigns this list to the variable **OBJECTS**.

Example 7 - continued

```
$(EXECUTABLE) : $(OBJECTS)
	$(FC) $^ -o $@
```

Here we have a generic reusable rule for linking an executable

Combined with the suffix.rules no source files needs to be specified in the makefile

Pattern rules

- Similar to suffix rules.
- No need to use .SUFFIXES for additional supported suffixes
- The recommended way in GNU Make
- Uses the % operator in the same way as in the wildcard functions

Example 8 – Generic make

```
FC=gfortran
FFLAGS=-c
EXECUTABLE=myprog
F90_FILES := $(wildcard *.f90)
OBJECTS := $(patsubst %.f90, %.o, $(F90_FILES))

$(EXECUTABLE) : $(OBJECTS)
	$(FC) $^ -o $@

%.o: %.f90
	$(FC) $(FFLAGS) $< -o $@

mymodule.o : myutils.o

clean :
	rm -rf *.o *.mod $(EXECUTABLE)
```

No explicit specification of source files or object-files

Modules dependencies still needs to be handled.

Maintaining Fortran projects with CMake

CMake

- CMake is a cross-platform build system
- Generates different types of build files
  - Makefiles of different flavors
  - Project files for Eclipse, Visual studio and Xcode
- Is a language

Example 1

```
cmake_minimum_required(VERSION 3.20)

project(simple)

enable_language(Fortran)

add_executable(simple myprog.f90)
```

CMake version control

Name of project (not executable)

Activate Fortran support. Not on by default.

Create an executable target, simple, that uses myprog.f90 as source.

Example 1 – cont...

```
$ ls
CMakeLists.txt myprog.f90
$ cmake .
-- The C compiler identification is GNU 4.2.1
-- The CXX compiler
        ...
-- Configuring done
-- Generating done
-- Build files have been written to: /Users/.../simple
$ ls
CMakeCache.txt
CMakeFiles Makefile myprog.f90
```

Running make

```
$ make
Scanning dependencies of target simple
[100%] Building Fortran object
CMakeFiles/simple.dir/myprog.f90.
o
Linking Fortran executable simple
[100%] Built target simple
```

Example 1 – Using a build directory

CMake generates a large number of build files. To avoid mixing these with existing source. A build, directory is often used.

```
$ mkdir build
$ cd build
$ cmake ..
-- The C compiler identification is GNU 4.2.1 .
.
-- Generating done
-- Build files have been written to: /Users/.../simple/build
```

Debug and release versions

- Cmake supports building for debugging and a release version
- The **CMAKE_BUILD_TYPE** variable controls the current build type
  - Set by using the -D option on the command line

```
$ cmake -D CMAKE_BUILD_TYPE=Release ..
-- Configuring done
-- Generating done
-- Build files have been written to: /Users/lindemann/
Development/progsci_book/source/cmake_examples/simple/b
```

Library dependencies

```
cmake minimum required(VERSION 3.20)

project(simple)

enable language(Fortran)

add_executable(simple myprog.f90)
target_link_libraries(simple blas m)
```



target_link_libraries associates a target with its library dependencies. Same notation used for both Unix and Windows builds.

Conditional builds

- To control configuration and build options for different platforms. Conditional build statements can be used.
- Predefined variables for different platforms exists (UNIX, LINUX, APPLE, WIN32)

```
if (UNIX)
       message(”This is a Unix build.”)
endif (UNIX)
```

Running CMAke on the above code on a Unix based system will print the message on standard output.

Conditional builds

```
if (UNIX)
       add_executable(multiple myprog.f90 mymodule.f90)
       target_link_libraries(multiple blas m)
else (UNIX)
       if (WIN32)
              add_executable(multiple myprog.f90 mymodule.f90)
              target_link_libraries(multiple blas32)
       else (WIN32)
              message(”Not supported configuration.”)
       endif (WIN32)
endif (UNIX)
```

Variables

- Variables are defined using the **set** command
  - set(MYVAR "This is a variable")
- Using the variables value requires it to be enclosed in ${...}
  - message(${MYVAR})
- Lists are also created using the **set** command
  - set(MYLIST a b c)

Iterating over lists

```
set(MYLIST a b c)
Foreach(i ${MYLIST})
       message(${i})
Endforeach(i)
```

Running CMake

```
$ cmake ..
a
b
c
-- Configuring done
-- Generating done
-- Build files have been written to: ...
```

Controlling optimisation

- The optimisation options for Fortran are controlled using the variables:
  - CMAKE_Fortran_FLAGS_RELEASE
  - CMAKE_Fortran_FLAGS_DEBUG
- The chosen Fortran compiler can be set using the variable
  - CMAKE_Fortran_COMPILER

CMake and compiler options

```
get_filename_component (Fortran_COMPILER_NAME ${CMAKE_Fortran_COMPILER} NAME)

if (Fortran_COMPILER_NAME STREQUAL "gfortran")
  set (CMAKE_Fortran_FLAGS_RELEASE "-funroll-all-loops -fno-f2c -O3")
  set (CMAKE_Fortran_FLAGS_DEBUG   "-fno-f2c -O0 -g")
elseif (Fortran_COMPILER_NAME STREQUAL "ifort")
  set (CMAKE_Fortran_FLAGS_RELEASE "-f77rtl -O3")
  set (CMAKE_Fortran_FLAGS_DEBUG   "-f77rtl -O0 -g")
elseif (Fortran_COMPILER_NAME STREQUAL "g77")
  set (CMAKE_Fortran_FLAGS_RELEASE "-funroll-all-loops -fno-f2c -O3 -m32")
  set (CMAKE_Fortran_FLAGS_DEBUG   "-fno-f2c -O0 -g -m32")
else (Fortran_COMPILER_NAME STREQUAL "gfortran")
  message ("No optimized Fortran compiler flags are known.")
  set (CMAKE_Fortran_FLAGS_RELEASE "-O2")
  set (CMAKE_Fortran_FLAGS_DEBUG   "-O0 -g")
endif (Fortran_COMPILER_NAME STREQUAL "gfortran")
```

CMake – Project generation

- CMake supports the generation of project files for most development environments
  - Visual Studio
  - Eclipse
  - NetBeans
  - Xcode
  - And more
- The –G command line option is used to select the used generator (default is makefiles)

CMake – Project generation

```
mkdir build_eclipse
$ cd build_eclipse/
$ cmake -G "Eclipse CDT4 - Unix Makefiles" ../multiple/
-- The C compiler identification is GNU 4.2.1
-- The CXX compiler identification is Clang 4.0.0
-- Could not determine Eclipse version, assuming at least 3.6 (
Helios). Adjust CMAKE_ECLIPSE_VERSION if this is wrong. ...
-- Generating done
-- Build files have been written to: ...
$ ls -la
total 112
drwxr-xr-x   8 lindemann  staff    272 Aug 29 20:07 .
drwxr-xr-x  13 lindemann  staff    442 Aug 29 20:06 ..
-rw-r--r--   1 lindemann  staff  14343 Aug 29 20:07 .cproject
-rw-r--r--   1 lindemann  staff   5527 Aug 29 20:07 .project
-rw-r--r--   1 lindemann  staff  17808 Aug 29 20:07 CMakeCache.txt
drwxr-xr-x  21 lindemann  staff    714 Aug 29 20:07 CMakeFiles
-rw-r--r--   1 lindemann  staff   4770 Aug 29 20:07 Makefile
-rw-r--r--   1 lindemann  staff   1562 Aug 29 20:07 cmake_install.cmake
```

These files can be opened directly in Eclipse